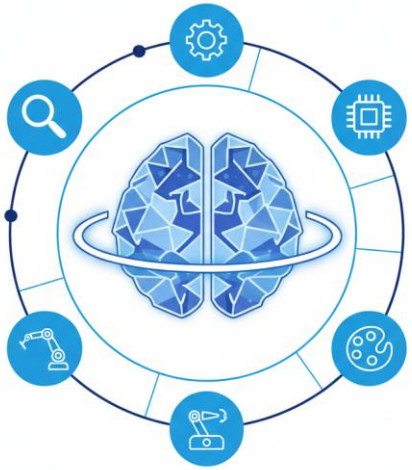
데이터 분석 및 시각화

pandas / matplotlib



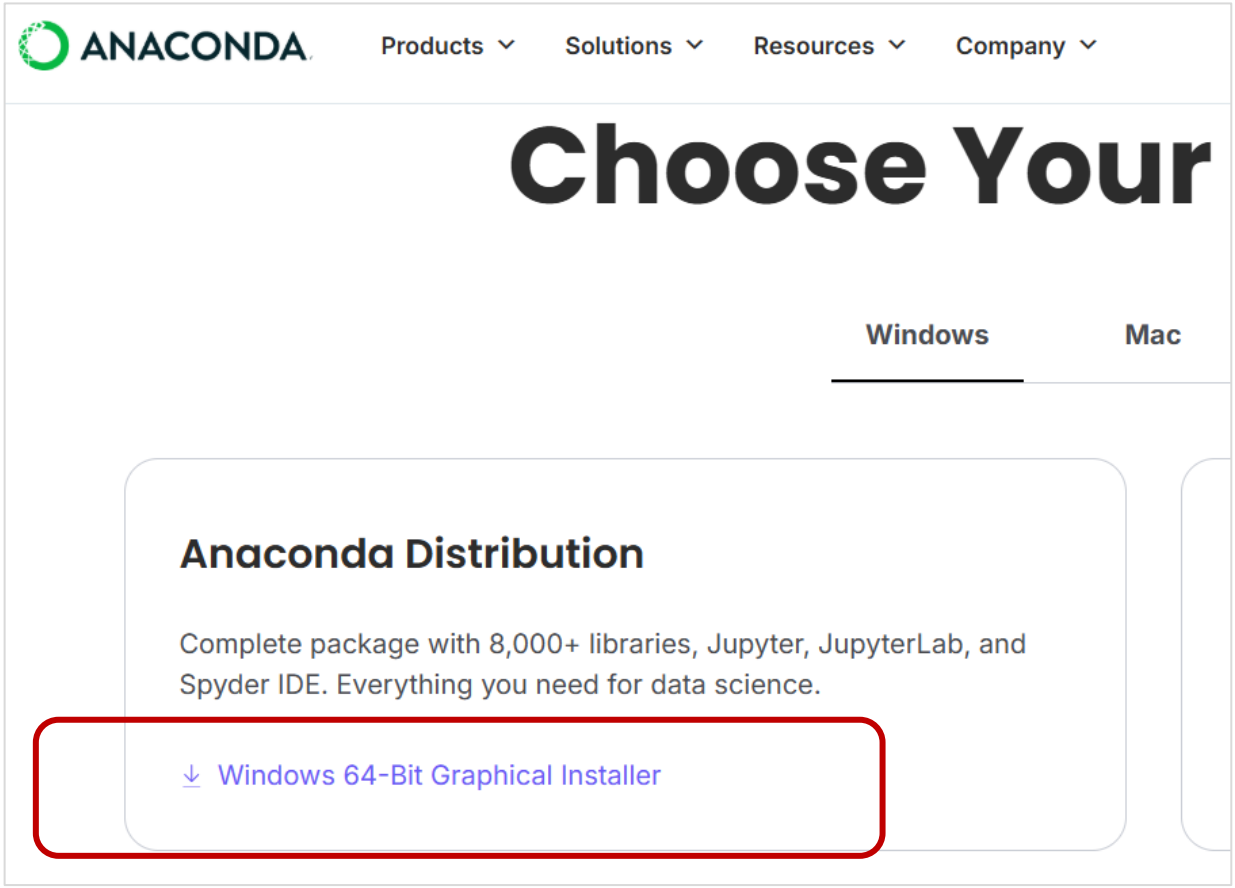
데이터 분석

- **데이터 분석 및 머신러닝 주요 IDE**

구분	주피터 노트북 (Jupyter)	구글 코랩 (Colab)
실행 환경	로컬 PC (내 컴퓨터)	클라우드 (구글 서버)
설치 여부	필요 (Anaconda 등)	불필요 (브라우저 실행)
인터넷 연결	오프라인 가능	반드시 필요
컴퓨팅 자원	내 PC 사양에 의존	구글 자원 사용 (GPU 무료 제공)
가격	무료	무료 (유료 Pro 버전 존재)
협업	파일 공유(ipynb) 필요	공유 링크로 즉시 협업 가능

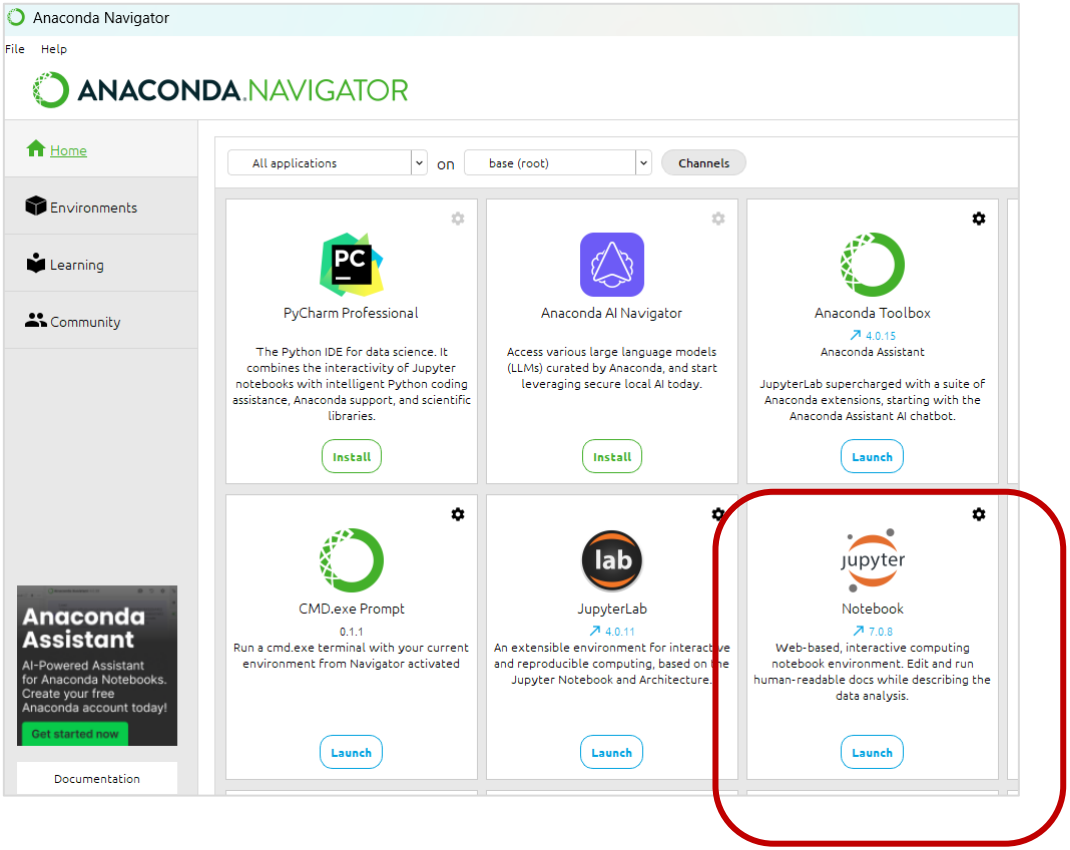
아나콘다(Anaconda) 플랫폼

❖ **Anaconda Download**



아나콘다(Anaconda) 플랫폼

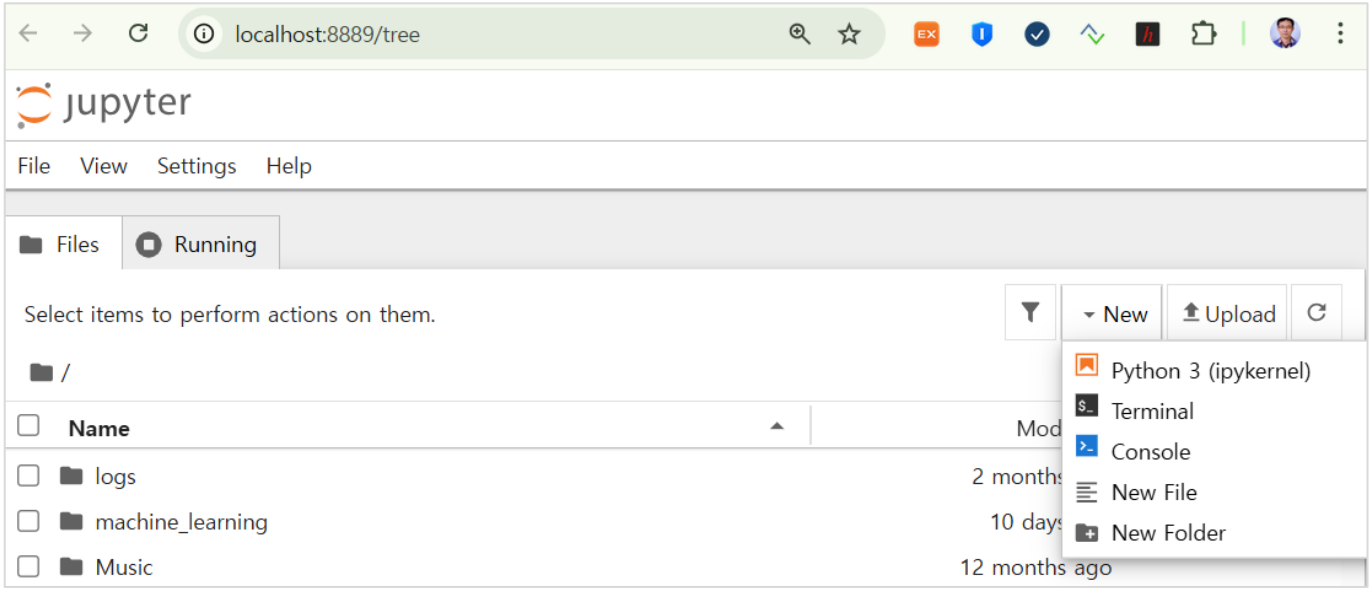
❖ 주피터 노트북(Jupyter Notebook) IDE 실행

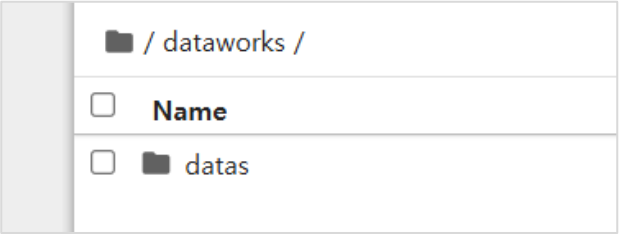


주피터 노트북(Jupyter Notebook)

❖ 작업 디렉터리 생성

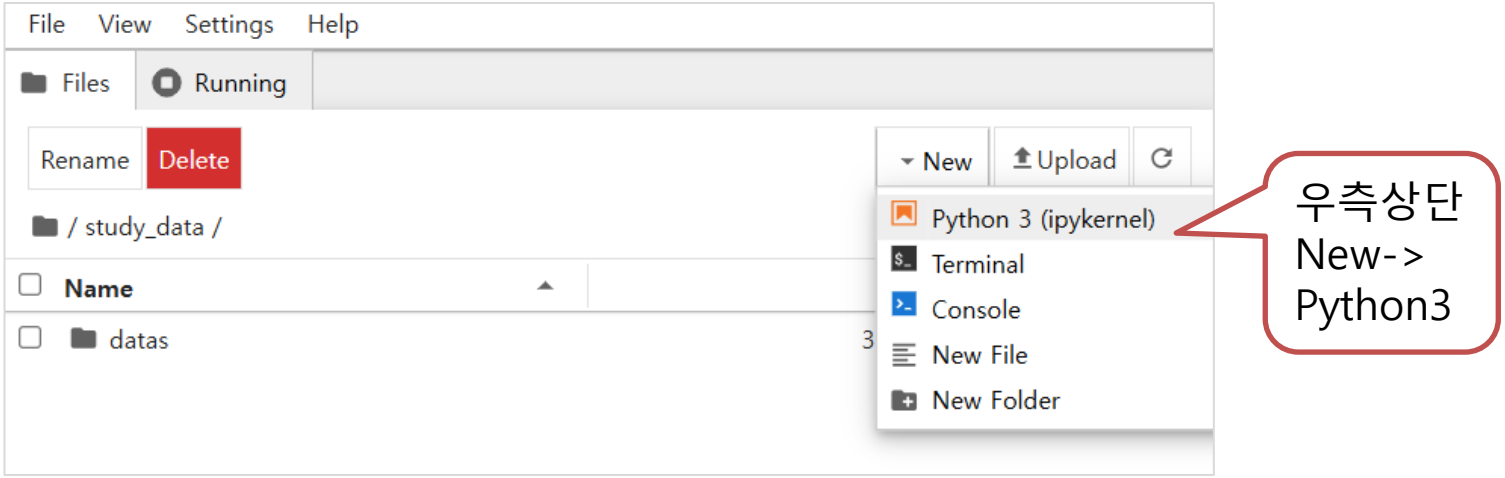
New > New Folder > dataworks 폴더 생성 > 하위에 datas 폴더 생성

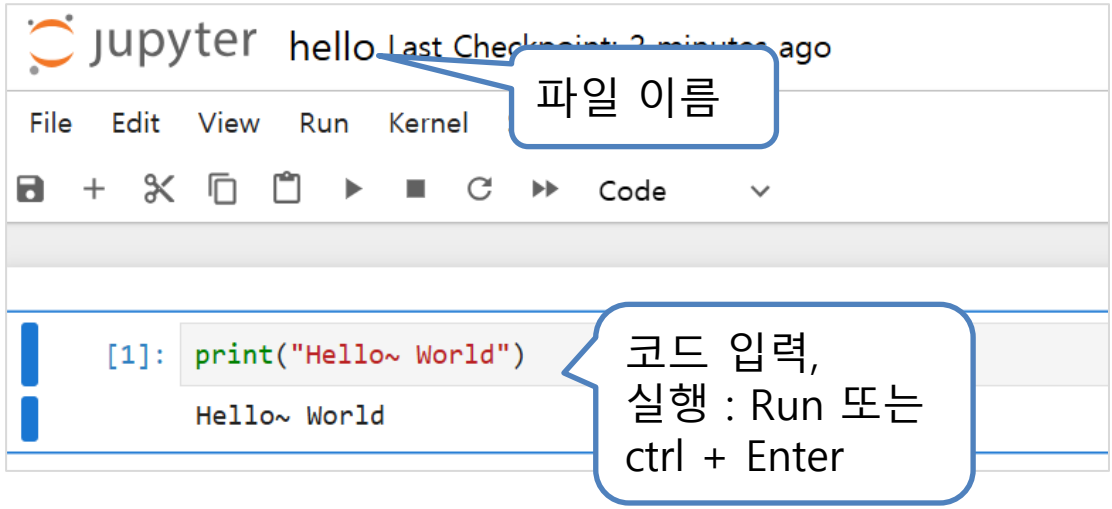




주피터 노트북(Jupyter Notebook)

❖ 파일 생성(.ipynb)





주피터 노트북(Jupyter Notebook)

❖ 코드 입, 출력

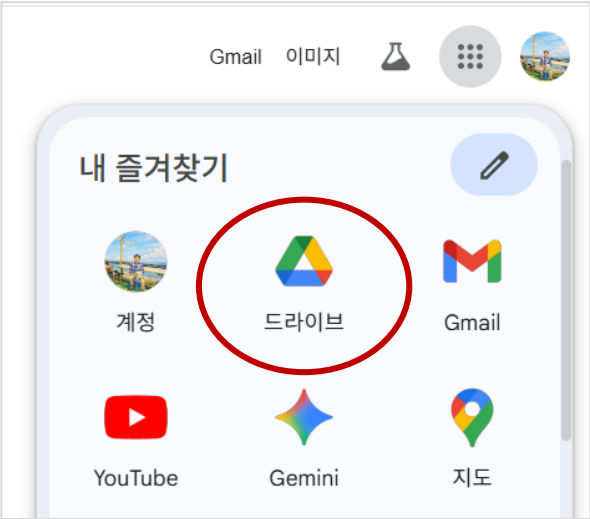
```
[1]: print("Hello~ World")

     Hello~ World

[2]: print("안녕~ 세계")

     안녕~ 세계

[7]: 10 + 20
     10 - 20

[7]: -10
```

```
[6]: n1 = 20
     n2 = 10

     print(n1 + n2)
     print(n1 - n2)
     print(n1 * n2)
     print(n1 / n2)
     print(n1 % n2)

     30
     10
     200
     2.0
     0
```

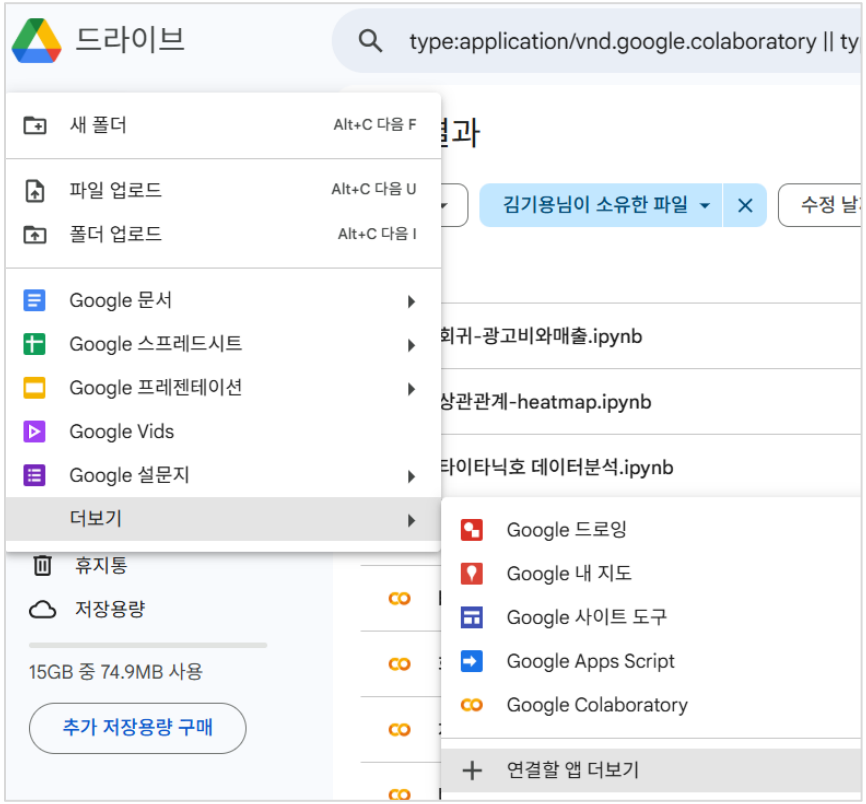
```
[10]: carts = ["계란", "라면", "콩나물"]
      print(carts)

      for cart in carts:
          print(cart)

      ['계란', '라면', '콩나물']
      계란
      라면
      콩나물
```

구글 코렙

❖ 구글 드라이브 > 코렙 설치하기



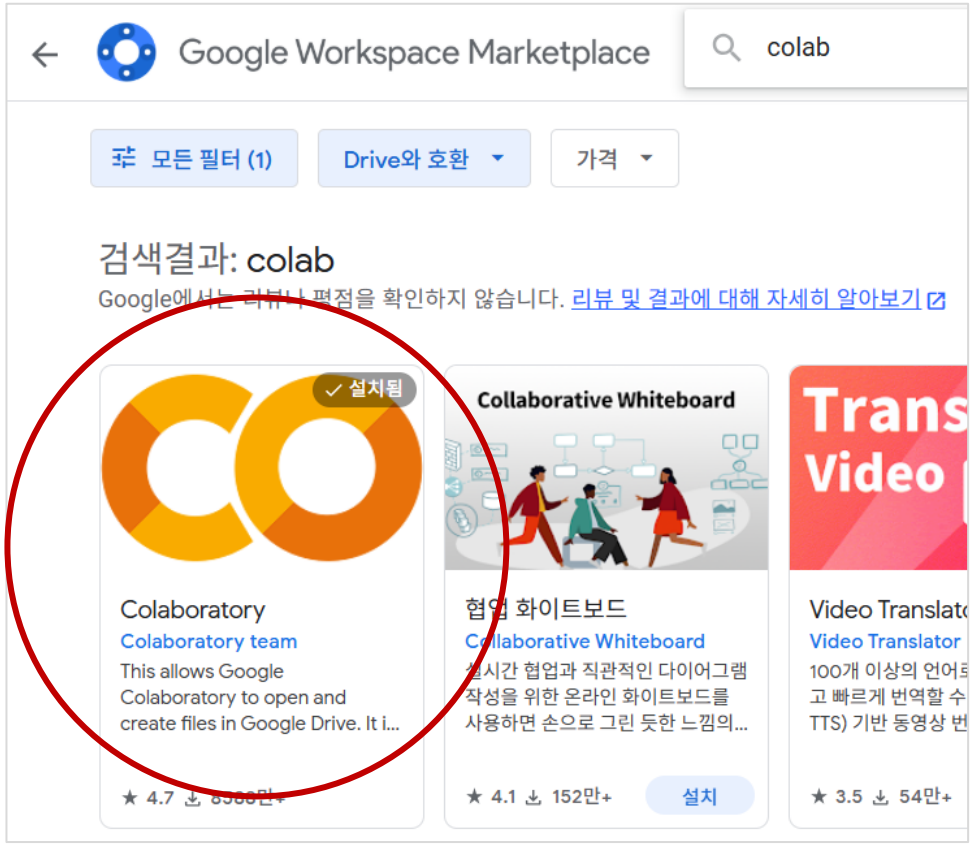
- + 신규 > 연결할 앱 더보기



구글 코렙

❖ 구글 드라이브 > 코렙 설치하기

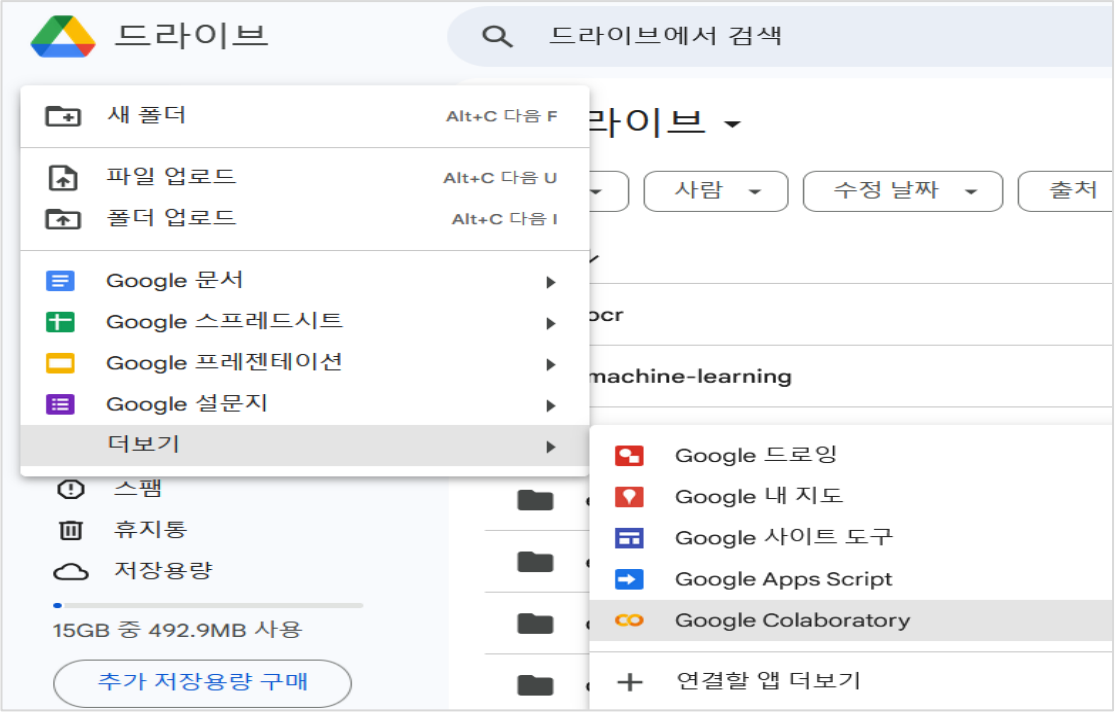
- 연결할 앱 더보기 > colab 검색 > 코렙 설치



구글 코렙

❖ 새 노트 만들기

- + 신규 > 더보기 > Google Colaboratory

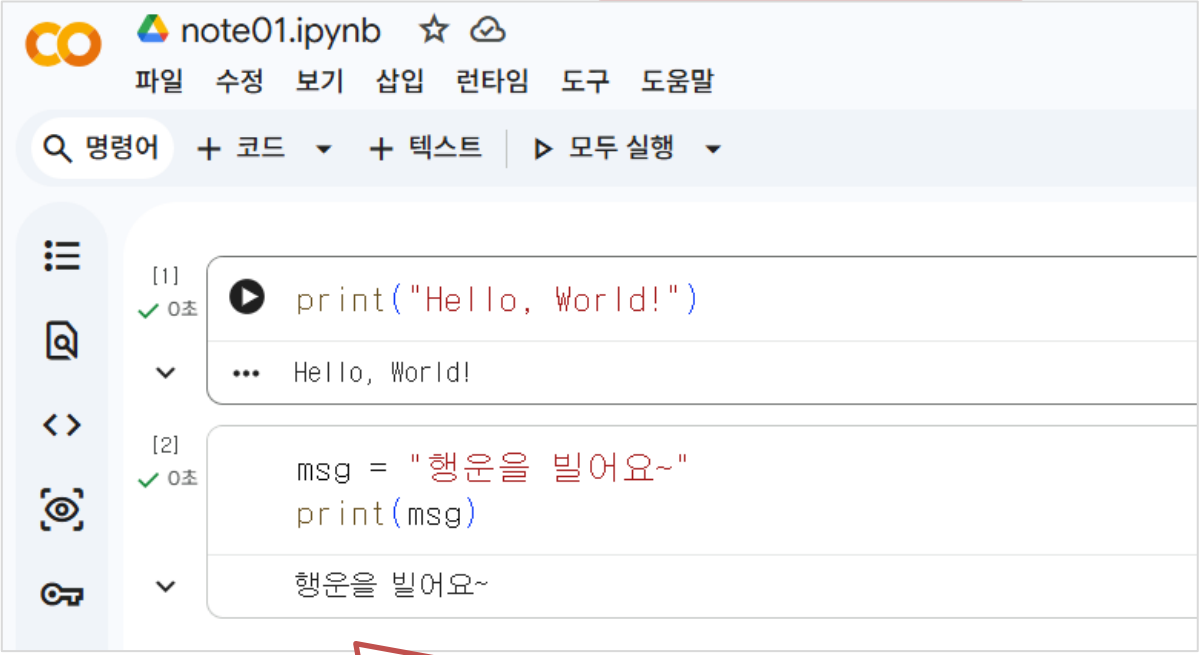


구글 코렙

❖ 새 노트 만들기

- 파일이름 > note01.ipynb

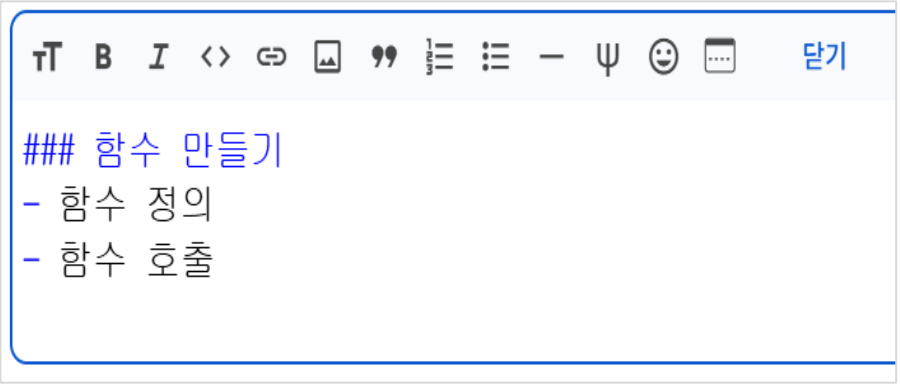
확장자는 .ipynb



코드 실행 : 실행버튼 또는 ctrl + Enter

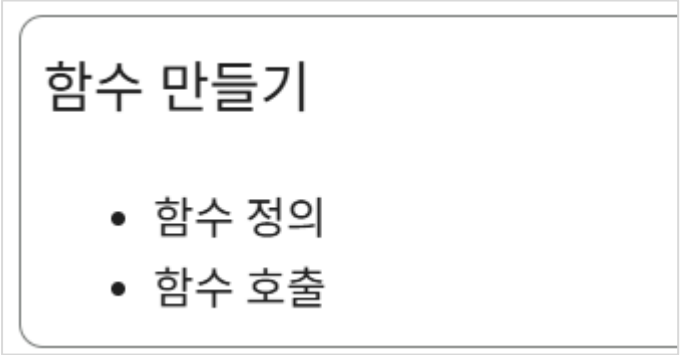
❖ 텍스트 사용하기

- 텍스트(버튼) > 문서의 제목이나 소제목을 만들때 사용



<편집화면>

shift + enter



<출력화면>

구글 코렙

❖ 코드 추가

+ 코드 + 텍스트

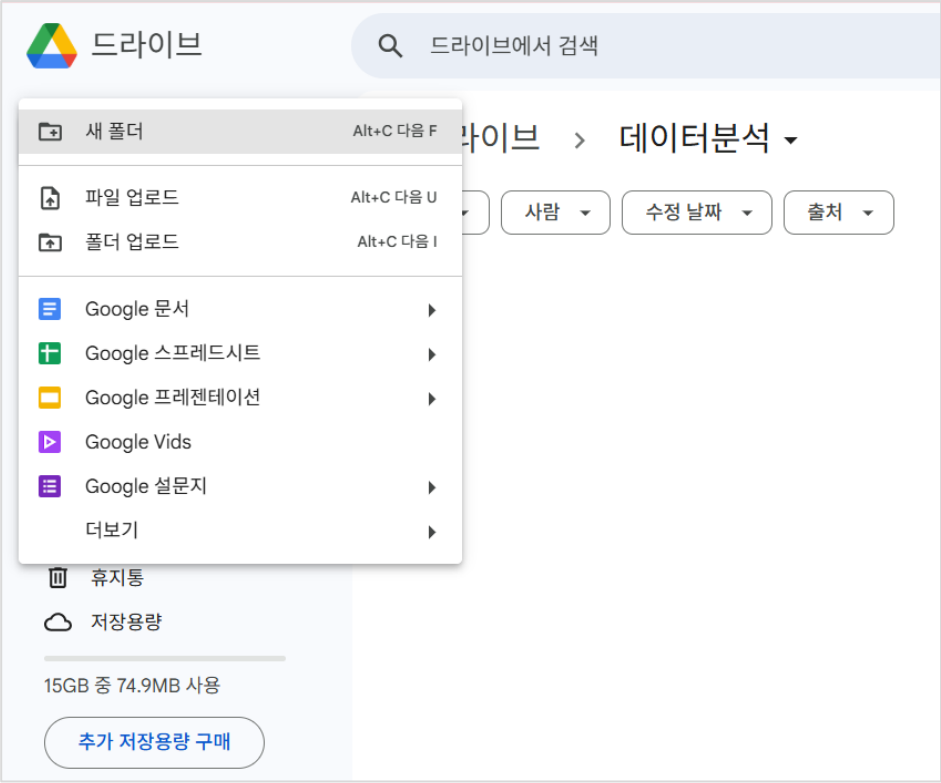
```python
# 절대값 계산 함수 정의
def my_abs(x):
  if x < 0:
    return -x
  else:
    return x

print(my_abs(-12))
```

```
12
```

구글 코렙

❖ 새 폴더 만들기 – 이름: 데이터분석

- + 신규 > 새폴더



데이터 분석

- **데이터 분석용 주요 라이브러리**

+ 데이터 분석을 위한 주요 라이브러리를 알려줘

라이브러리	주요 역할	핵심 기능
pandas	데이터 분석 핵심	DataFrame, CSV/Excel 처리, 필터링, 그룹화
numpy	수치계산 기반	다차원 배열, 수학/통계 연산, 고속 연산
matplotlib	데이터 시각화	선/막대/산점도 그래프

데이터 분석

- **판다스**
  - 데이터를 수집, 정리하는 데 최적화된 도구로 **표 형태(행·열)의 데이터**를 빠르고 편리하게 처리하기 위한 라이브러리입니다.
  - 가장 인기있는 프로그래밍 언어인 **파이썬(python)**을 기반으로 한다.
  - 시리즈(Series)와 데이터프레임(DataFrame)이라는 구조화된 데이터 형식을 제공한다.
  - 판다스 설치 - **아나콘다 플랫폼**에는 이미 설치되어 있습니다.

    (pip install pandas)

데이터 분석

- **판다스 자료구조**
  - **시리즈(Series)** – 데이터가 순차적으로 나열된 1차원 배열의 형태를 가지며, 인덱스(Index)는 데이터 값(value)과 일대일 대응이 된다.
  - **데이터프레임(DataFrame)** – 행(Row)과 열(Column)로 만들어지는 2차원 배열 구조로 표(Table) 형태이다.

	제품명
0	키보드
1	마우스
2	USB 메모리

인덱스 (index)

[시리즈]

	제품명	가격
0	키보드	20,000
1	마우스	32,000
2	USB 메모리	10,000

칼럼 (column)

[데이터프레임]

데이터 분석

- **Series(시리즈)**

import pandas as pd(별칭)

pd.Series(데이터, 인덱스)

속성	기능
index	- 시리즈의 인덱스 (생략하면 기본 0, 1, 2)
values	- 시리즈의 값(요소)
shape	- 시리즈의 크기 (요소의수,)

데이터 분석

- **Series(시리즈)**

```
import pandas as pd

data = ['키보드', '마우스', 'USB 메모리']

# Series 클래스로 s1 데이터 생성
s1 = pd.Series(data)
# s1
print(s1)
print(type(s1))

# Series 속성
print(s1.index)
print(s1.values)
print(s1.shape) # 튜플

# 요소 선택(검색)
print(s1[0])
print(s1[1])
print(s1[2])
```

데이터 분석

- **DataFrame(데이터프레임)**

```
import pandas as pd(별칭)

pd.DataFrame(데이터, 인덱스)
```

속성	기능
index	- 데이터프레임의 인덱스 (생략하면 기본 0, 1, 2)
columns	- 데이터프레임의 칼럼
shape	- 데이터프레임의 크기 (행, 열)

데이터 분석

- **DataFrame(데이터프레임)**

```
import pandas as pd

df = pd.DataFrame({
    "name": ['키보드', '마우스', 'USB 메모리'],
    "price": [20000, 32000, 11000]
})

df
df.columns #['name', 'price']
df.shape  #(3, 2)
```

데이터 분석

- **DataFrame(데이터프레임)**

```
# 칼럼 추가
df['spec'] = ['유선', '무선', 'USB 메모리']
df

# 칼럼명 변경 - rename()
df = df.rename(columns={
    "name": "제품명",
    "price": "가격",
    "spec": "제품상세"
    })
df
```

	제품명	가격	제품상세
0	키보드	20000	유선
1	마우스	32000	무선
2	USB 메모리	11000	USB 메모리

데이터 분석

- **제미나이에게 csv 질문하기**

csv 파일에 대해 설명해 주세요

+ Gemini 2.5 Flash

Gemini

csv 파일에 대해 설명해 주세요

CSV(Comma Separated Values) 파일은 데이터를 쉼표로 구분하여 저장하는 텍스트 파일 형식입니다. 스프레드시트나 데이터베이스에서 데이터를 저장하고 교환할 때 널리 사용됩니다. 각 줄은 데이터베이스 테이블의 한 행을 나타내고, 각 줄의 쉼표는 해당 행의 다른 필드를 구분합니다. CSV 파일은 일반 텍스트 파일이므로 어떤 텍스트 편집기로든 열 수 있고, 다양한 프로그램에서 쉽게 가져오거나 내보낼 수 있다는 장점이 있습니다.

데이터 분석

- **데이터 프레임을 CSV 파일로 변환**

  CSV(Comma Seperated Value) 파일: 쉼표(,)로 데이터를 구분하여 저장하는 텍스트 파일 형식이다. 주로 표형태의 데이터를 저장하는 데 사용하며 엑셀에서 쉽고 열고 편집할 수 있음

☞ CSV 파일로 변환

df.to_csv("computer.csv", index=False)

☞ CSV 파일 읽기

pd.read_csv("computer.csv")

데이터 분석

- **CSV 파일로 변환 및 CSV 읽기**

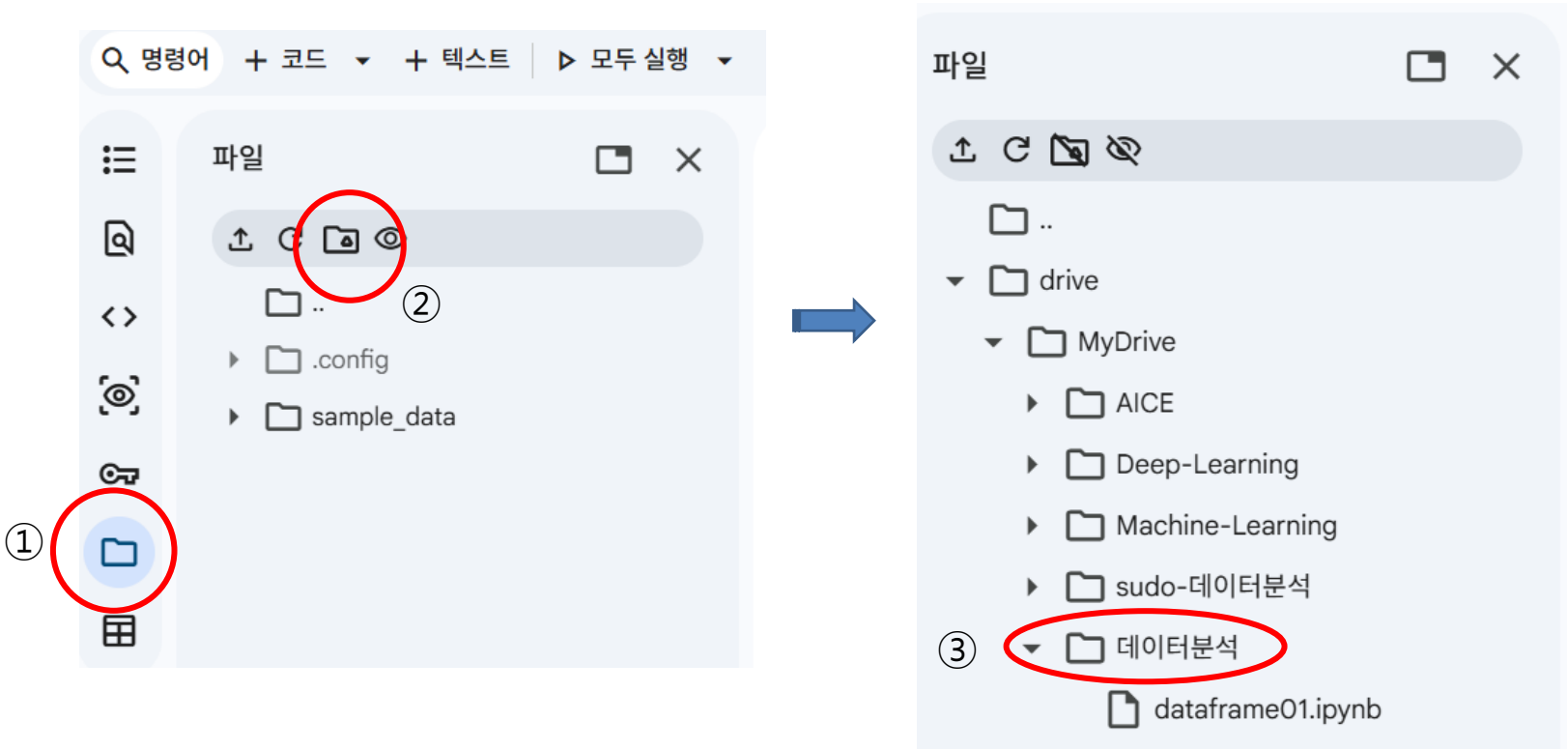
```
# csv 파일 만들기
df.to_csv("/content/drive/MyDrive/데이터분석/computer.csv", index=False)

# csv 파일 읽기
computer = pd.read_csv("/content/drive/MyDrive/데이터분석/computer.csv")
computer
```

	제품명	가격	제품상세
0	키보드	20000	유선
1	마우스	32000	무선
2	USB 메모리	11000	USB 메모리

구글 코렙

❖ 드라이브 마운트

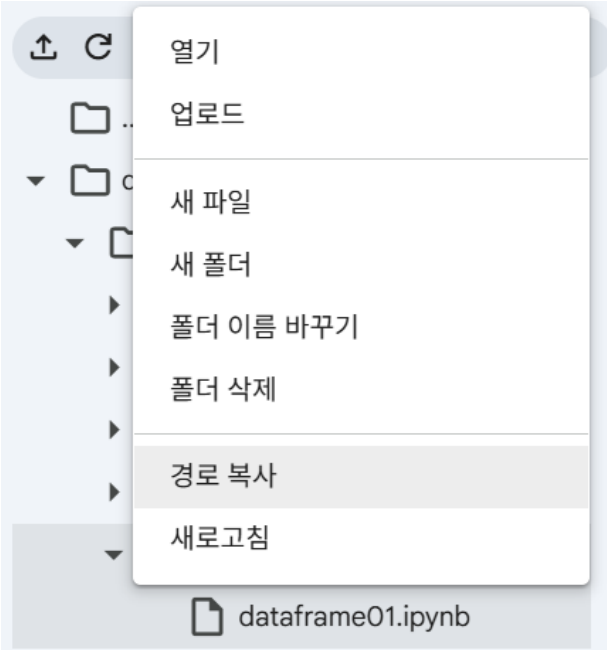
- 폴더 > 드라이브 마운트 > 드라이브 연결

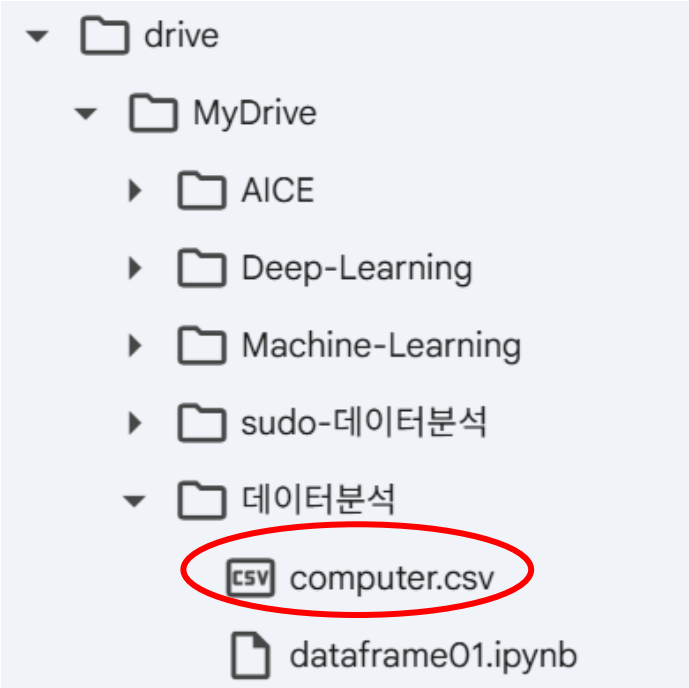


구글 코렙

❖ 훈련 데이터 업로드

- (...) 메뉴 > 경로 복사





데이터 관리

- **행과 열 선택**

속성	범위	예시
loc[인덱스명, 칼럼명] (location)	끝 인덱스 포함	인덱스 - 라벨인덱스
iloc[인덱스번호, 칼럼번호] (integer location)	끝 인덱스 - 1	인덱스 – 순서(0부터 시작)

데이터 관리

- **행/열 선택 : loc[행이름, 열이름] 사용**

```
# 행 선택 - loc[행(행 이름)]
df.loc[0:1] #0행, 1행 출력

# 특정 요소 선택 - loc[행 이름, 열 이름]
print(df.loc[0, "제품명"]) # 키보드
print(df.loc[1, "가격"]) # 32000
print(df.loc[0:1, "제품명"])
print(df.loc[0:1, ["제품명", "가격"]])
```

```
키보드
32000
0    키보드
1    마우스
Name: 제품명, dtype: object
   제품명     가격
0  키보드  20000
1  마우스  32000
```

데이터 관리

- **행/열 선택 : iloc[행번호, 열번호] 사용**

```
# 행 선택 - iloc[행(인덱스)]
df.iloc[0:1] #0행만 출력(첫인덱스: 끝인덱스-1)

# 특정 요소 선택 - iloc[행 인덱스, 열 인덱스]
print(df.iloc[0, 0]) # 키보드
print(df.iloc[1, 1]) # 32000
print(df.iloc[0:2, 0])
print(df.iloc[0:2, [0, 1]])
```

```
키보드
32000
0    키보드
1    마우스
Name: 제품명, dtype: object
   제품명     가격
0  키보드  20000
1  마우스  32000
```

데이터 관리

- 데이터 프레임 - 요소값 변경(업데이트)

df.loc[행이름, 열이름] = “변경할 값 "

df.iloc[행번호, 열번호] = “변경할 값 "

- 데이터 프레임 - 요소값 추가

df[열] = “추가할 칼럼“ => 열 추가

df.loc[행] = “추가할 값“ => 행 추가

데이터 관리

- **행/열 요소 변경 및 추가**

```
# 요소값 변경 및 추가
df2 = df.copy() # 원본 복사
df2

# '무선마우스'의 가격을 27000원으로 변경
df2.loc[1, "가격"] = 27000

# 행 추가
df2.loc[3] = ["키보드", 30000, "무선"]
df2

# 열 추가 - '할인가' (가격 * 10%)
df2["할인가"] = df2['가격'] * 0.1
df2
```

	제품명	가격	제품상세	할인가
0	키보드	20000	유선	2000.0
1	마우스	27000	무선	2700.0
2	USB 메모리	11000	USB 메모리	1100.0
3	키보드	30000	무선	3000.0

데이터 관리

- **행/열 삭제**

df.drop(행, axis=0) => 행 삭제

df.drop(열, axis=1) => 열 삭제

```
# 행, 열 삭제
df3 = df2.copy()
df3

# 3행 삭제
df3 = df3.drop(3, axis=0)

# '할인가' 열 삭제
df3 = df3.drop("할인가", axis=1)
df3
```

데이터 탐색

- **데이터 탐색을 위한 주요 함수**

```
# 데이터 정보
df2.info()

# 데이터 출력 - 기본 5개
df2.head(2)

# 통계 요약 - 기본 숫자형 칼럼
df2.describe()
df2.describe(include='number')

# 문자형 칼럼 요약
df2.describe(include='object')

# 고유값의 개수
df2["제품명"].value_counts()
```

데이터 탐색

- **데이터 탐색을 위한 주요 함수**

```
<class 'pandas.core.frame.DataFrame'>
Index: 4 entries, 0 to 3
Data columns (total 4 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   제품명      4 non-null      object
 1   가격       4 non-null      int64
 2   제품상세     4 non-null      object
 3   할인가      4 non-null      float64
dtypes: float64(1), int64(1)
memory usage: 332.0+ bytes
```

	가격	할인가
count	4.000000	4.000000
mean	22000.000000	2200.000000
std	8445.906306	844.590631
min	11000.000000	1100.000000
25%	17750.000000	1775.000000
50%	23500.000000	2350.000000
75%	27750.000000	2775.000000
max	30000.000000	3000.000000

제품명	count
키보드	2
마우스	1
USB 메모리	1

데이터 전처리

- **결측치 삭제 및 대체**

함수	설명
drop(행, axis=0) drop(열, axis=1)	NaN이 포함된 행 삭제 NaN이 포함된 열 삭제
dropna()	NaN이 포함된 모든 행 삭제
fillna()	- 수치형: 평균(mean()), 중간값(median()) - 문자형: 빈도값(mode())
isnull() isnull().sum()	- 결측치 확인(True / False로 반환) - 결측치 개수(True(1))

데이터 전처리

- **결측치 대체**

```
import pandas as pd

data = {
    "메뉴": ["김밥", "비빔밥", "자장면", "우동", None, "자장면", "김밥"],
    "가격": [3000, 7000, 5500, None, 3500, 6000, 4000],
    "분류": ["한식", "한식", "중식", "일식", "한식", "중식", "한식"]
}

food = pd.DataFrame(data)
food

# 데이터 탐색
food.info()
food.describe()
food.head()
```

데이터 전처리

- **결측치 대체**

```
# csv 파일 만들기
food.to_csv("/content/drive/MyDrive/데이터분석/food.csv", index=False)
```

```
# csv 파일 읽기
df = pd.read_csv("/content/drive/MyDrive/데이터분석/food.csv")
df
```

	메뉴	가격	분류
0	김밥	3000.0	한식
1	비빔밥	7000.0	한식
2	자장면	5500.0	중식
3	우동	NaN	일식
4	NaN	3500.0	한식
5	자장면	6000.0	중식
6	김밥	4000.0	한식

데이터 전처리

- **결측치 대체**

```
# 결측치 확인
df.isna().sum() #df.isnull().sum() 도 가능

# 결측치 대체 - 수치형(중간값)
median = df["가격"].median()
median #4750.0

# inplace=True, 즉시 변경
df["가격"].fillna(median, inplace=True)
df

# 결측치 확인
df.isnull().sum()
```

	0
메뉴	1
가격	0
분류	0

데이터 전처리

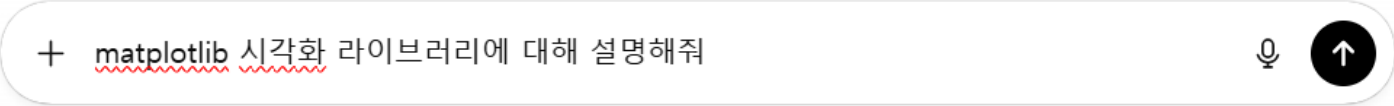
- **결측치 대체**

```
# 문자형(최빈값)
freq = df["메뉴"].mode()[0]
freq #김밥

df['메뉴'].fillna(freq, inplace=True)
df

# 결측치 확인
df.isnull().sum()
```

	0
메뉴	0
가격	0
분류	0

데이터 시각화

- **matplotlib 라이브러리**



- 데이터 분석, 과학 시각화, 보고서 작성 등 다양한 분야에서 그래프를 그릴 때 사용됩니다.
- 주요 그래프 종류

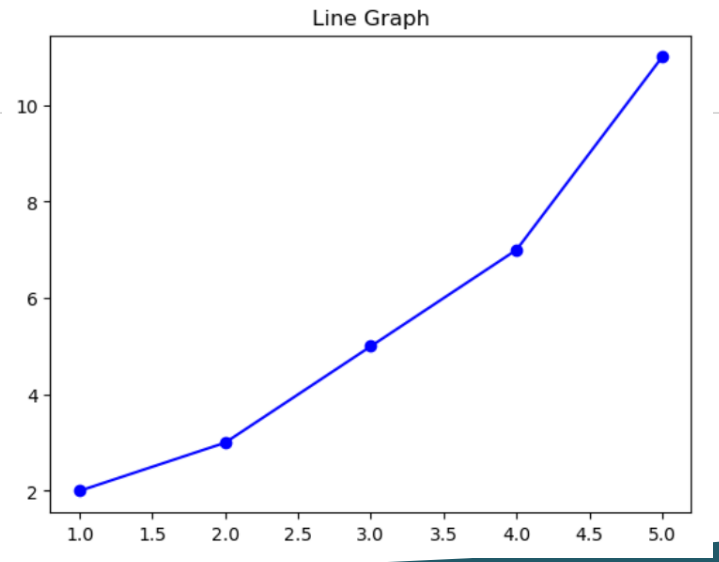
종류	함수명	예시
선 그래프	plot()	plt.plot(x, y)
막대 그래프(수직/수직)	bar() / barh()	plt.bar(x, y) / plt.barh(x, y)
산점도	scatter()	plt.scatter(x, y)
히스토그램	hist()	plt.hist(data)
파이차트	pie()	plt.pie(data)

41

데이터 시각화

- 선 그래프

```
import matplotlib.pyplot as plt

# 데이터 준비
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]

# 선 그래프
plt.plot(x, y, label='Prime numbers', color='blue', marker='o')
plt.title('Line Graph')
# plt.show()
```

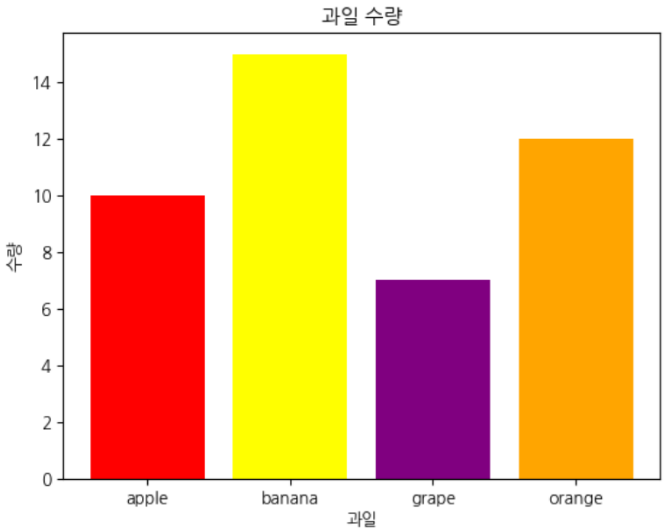


42

데이터 시각화

- 막대 그래프

```
import matplotlib.pyplot as plt

# 데이터 준비
fruits = ['apple', 'banana', 'grape', 'orange']
counts = [10, 15, 7, 12]
colors = ['red', 'yellow', 'purple', 'orange']

# 막대 그래프
plt.bar(fruits, counts, color=colors)
plt.title('과일 수량')
plt.xlabel('과일')
plt.ylabel('수량')

# plt.show()
```



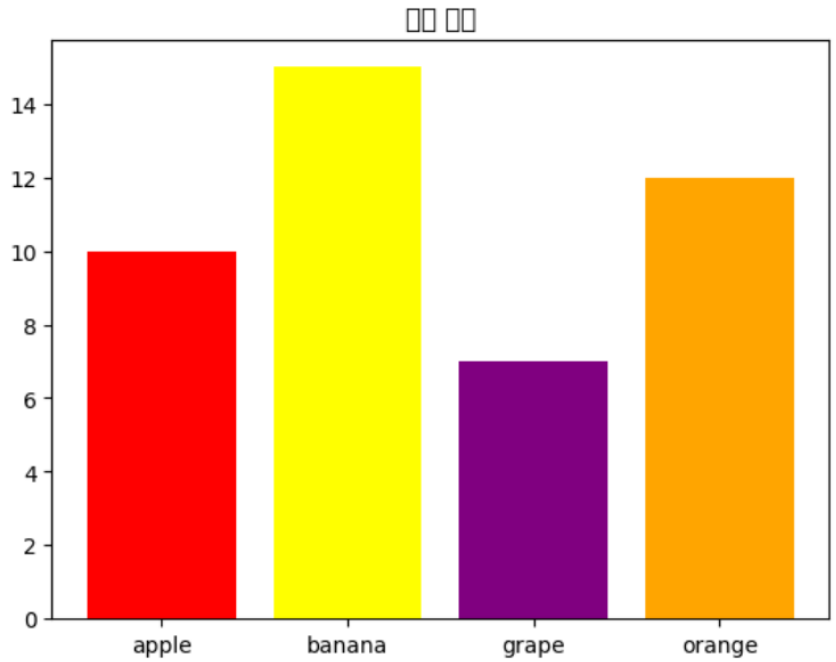
43

데이터 시각화

- 막대 그래프

막대 그래프의 제목을 한글로 표시해 주세요

+ Gemini 2.5 Flash



44

데이터 시각화

- 막대 그래프

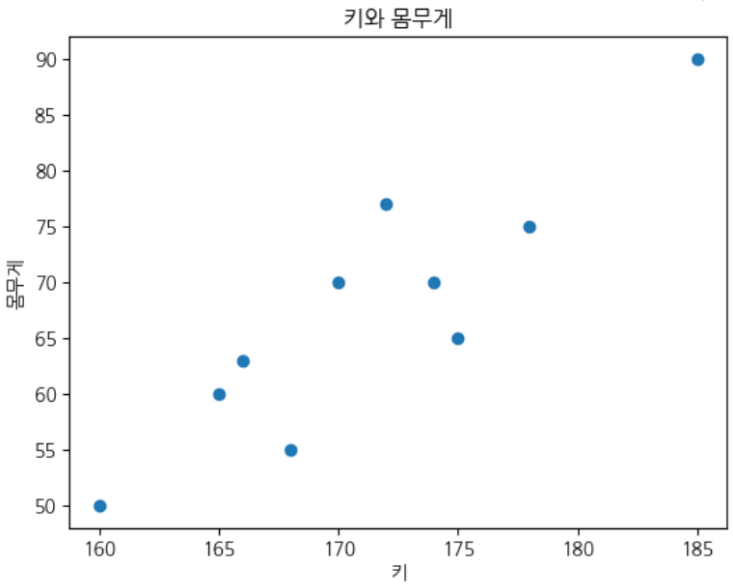
```
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm

# Install Korean font (NanumGothic)
!sudo apt-get install -y fonts-nanum
!sudo fc-cache -fv

# Set font path and configure matplotlib
font_path = '/usr/share/fonts/truetype/nanum/NanumGothic.ttf'
fm.fontManager.addfont(font_path)
plt.rc('font', family='NanumGothic')
plt.rcParams['axes.unicode_minus'] = False
```


45

데이터 시각화

- 산점도 그래프

```
# 데이터 준비
height = [160, 170, 175, 165, 172, 168, 174, 178, 185, 166]
weight = [50, 70, 65, 60, 77, 55, 70, 75, 90, 63]

# 산점도 그래프
plt.scatter(height, weight)
plt.title('키와 몸무게')
plt.xlabel('키')
plt.ylabel('몸무게')
```



46

상관 분석 VS 회귀 분석

- **상관 분석 vs 회귀 분석**

상관 분석	회귀 분석
x와 y의 관련 정도를 확인	x가 y에 주는 영향을 추정하여 결과를 예측할 수 있음
산포도, 상관 계수	회귀식
광고비와 매출은 양의 상관관계가 있다.	광고비를 1% 늘리면 매출이 5% 증가할 것이다.

47

seaborn 라이브러리

- **seaborn 라이브러리**
  - Python의 데이터 시각화 라이브러리로, 통계적 그래프를 쉽게 그릴 수 있도록 matplotlib을 기반으로 만들어졌다
  - pandas의 DataFrame과 잘 통합되어 있어, 데이터를 시각적으로 분석하고자 할 때 매우 유용하다

- **설치 방법**

```
pip install seaborn
```


48

자동차 사고 상관 분석

- **데이터 준비 및 탐색**

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 자동차 사고 데이터 준비
df = sns.load_dataset('car_crashes')
df.head()

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51 entries, 0 to 50
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   total           51 non-null     float64
 1   speeding        51 non-null     float64
 2   alcohol         51 non-null     float64
 3   not_distracted  51 non-null     float64
 4   no_previous     51 non-null     float64
 5   ins_premium     51 non-null     float64
 6   ins_losses      51 non-null     float64
 7   abbrev          51 non-null     object
```


49

자동차 사고 상관 분석

- **데이터 준비 및 탐색**

데이터셋에 포함된 각 칼럼의 의미를 알려줘

+ Gemini 2.5 Flash

변수	의미
total	10만 명당 총 사고 건수
speeding	과속 관련 사고 비율
alcohol	음주 관련 사고 비율
not_distracted	비산만 상태 사고 비율
no_previous	이전 사고 기록 없음 비율
ins_premium	보험료
ins_losses	보험 손실 비용
abbrev	주 약자 (문자형 → 상관분석 제외됨)

50

자동차 사고 상관 분석

- **상관 계수 및 시각화**

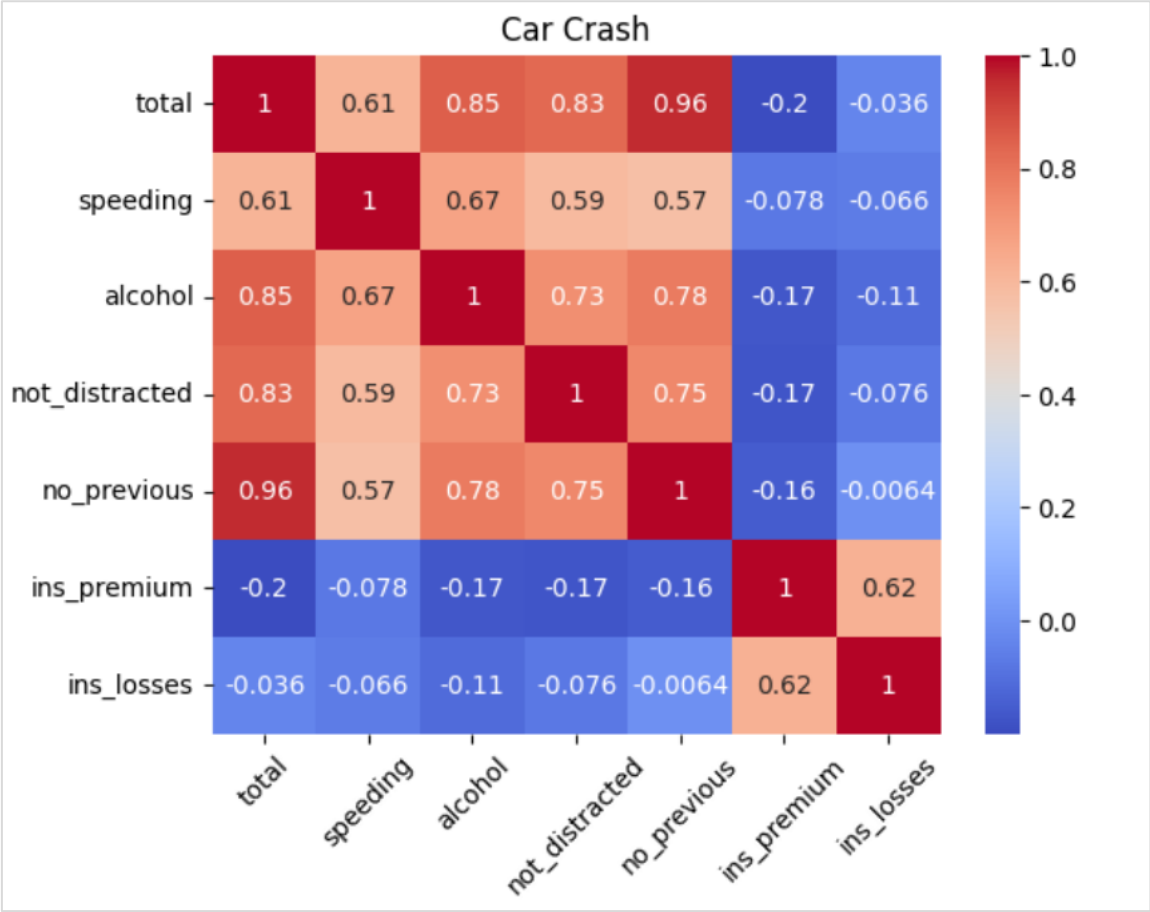
```
# 상관 계수
# corr = df.select_dtypes(include=['number']).corr()
corr = df.select_dtypes(exclude=['object']).corr()
corr

# 시각화 - 히트맵
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Car Crash')
plt.xticks(rotation=45)
```

```
(array([0.5, 1.5, 2.5, 3.5, 4.5, 5.5, 6.5]),
 [Text(0.5, 0, 'total'),
  Text(1.5, 0, 'speeding'),
  Text(2.5, 0, 'alcohol'),
  Text(3.5, 0, 'not_distracted'),
  Text(4.5, 0, 'no_previous'),
  Text(5.5, 0, 'ins_premium'),
  Text(6.5, 0, 'ins_losses')])
```


51

자동차 사고 상관 분석

- 상관 계수 및 시각화



52

자동차 사고 상관 분석

- **상관 계수 및 시각화**

상관 계수에 대해 설명해 줘

+ Gemini 2.5 Flash

상관 계수는 두 변수 간의 선형 관계의 강도와 방향을 측정하는 통계량입니다. 그 값은 -1과 1 사이이며, 각 값은 다음을 의미합니다:

- **1에 가까울수록 (양의 상관관계):** 두 변수가 같은 방향으로 움직이는 강한 경향이 있습니다. 한 변수가 증가하면 다른 변수도 증가합니다.
- **-1에 가까울수록 (음의 상관관계):** 두 변수가 반대 방향으로 움직이는 강한 경향이 있습니다. 한 변수가 증가하면 다른 변수는 감소합니다.
- **0에 가까울수록 (약한 또는 상관관계 없음):** 두 변수 사이에 선형 관계가 거의 없거나 전혀 없습니다. 한 변수의 변화가 다른 변수에 대한 예측을 거의 제공하지 않습니다.

예를 들어, 앞서 분석한 `corr` DataFrame에서 `total`과 `no_previous`의 상관 계수가 0.956으로 1에 매우 가깝다는 것은 이 두 변수 사이에 매우 강한 양의 선형 관계가 있음을 나타냅니다. 즉, 총 사고 건수가 많은 주에서는 이전 사고 기록이 없는 운전자가 관련된 사고 건수도 많을 가능성이 높다는 의미입니다.

53

자동차 사고 상관 분석

- **주요 상관 관계 분석**

변수 1	변수 2	상관계수	관계 강도	해석
total	no_previous	0.956	매우 강한 양의 상관관계	총 사고 건수가 많을수록 이전 사고 기록이 없는 운전자 관련 사고도 많음
total	alcohol	0.853	강한 양의 상관관계	총 사고 건수가 많을수록 음주 관련 사고도 많음
total	not_distracted	0.828	강한 양의 상관관계	총 사고 건수가 많을수록 부주의하지 않은 운전 관련 사고도 많음
total	speeding	0.612	보통 수준의 양의 상관관계	총 사고 건수가 많을수록 과속 관련 사고도 많음
total	ins_premium	-0.200	약한 음의 상관관계	총 사고 건수와 보험료는 약한 음의 관계
total	ins_losses	-0.036	매우 약한 음의 상관관계	총 사고 건수와 보험 손실은 거의 관계 없음
alcohol	no_previous	0.784	강한 양의 상관관계	음주 관련 사고율이 높은 주에서 이전 사고 기록이 없는 운전자 비율도 높은 경향
ins_premium	ins_losses	0.623	보통에서 강한 양의 상관관계	높은 보험료가 높은 보험 손실과 연관됨

54

머신러닝(Machine Learning)

+ 머신러닝에 대해 설명해 줘

머신러닝이란? 명시적으로 규칙을 코딩하지 않아도, 데이터로부터 패턴을 학습해 예측·판단하는 인공지능 기술

머신러닝의 핵심 개념

1️⃣ **데이터**

- 학습의 재료
- 많고, 정확할수록 성능이 좋아짐

2️⃣ **모델(Model)**

- 데이터를 학습한 결과물
- 예: "이메일이 스팸일 확률은 92%"

3️⃣ **학습(Training)**

- 데이터를 보고 모델이 규칙을 조정하는 과정

4️⃣ **예측(Prediction)**

- 새로운 데이터에 대해 결과를 맞혀보는 것

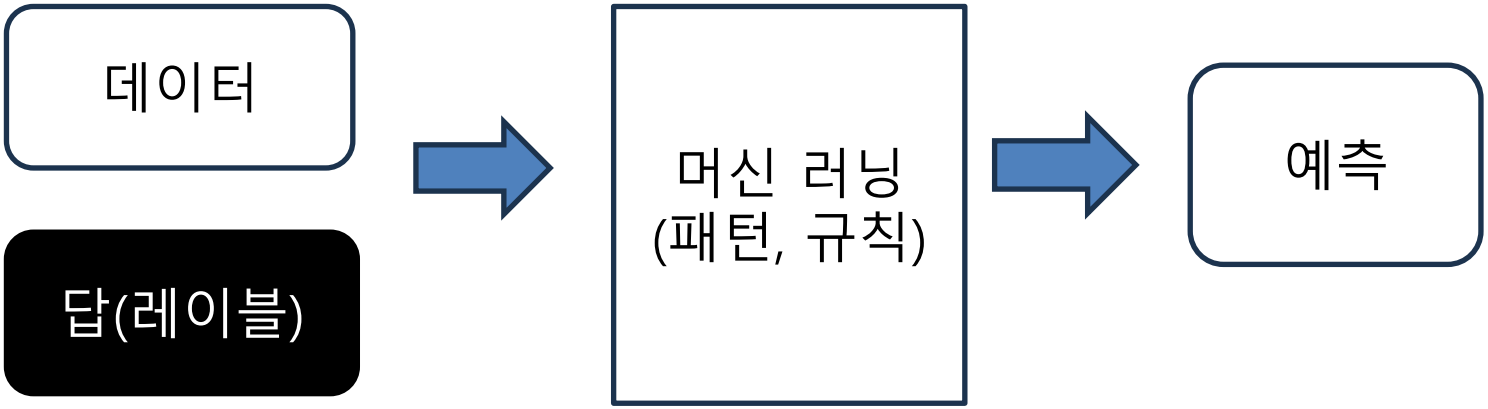
머신러닝(Machine Learning)

❖ 지도 학습 vs 비지도 학습

	지도 학습	비지도학습
분석 모형 (알고리즘)	• 회귀 • 분류	• 군집 • 차원 축소
특징	• 정답(레이블)을 알고 있는 상태에서 학습 • 회귀 – 가격, 매출, 주가 예측 • 분류 – 고객분류, 질병 진단, 스팸 메일 필터링	• 정답(레이블)이 없는 상태에서 서로 비슷한 데이터를 찾아서 그룹화(클러스터링) • 구매 패턴 분석, 유투브 추천 등

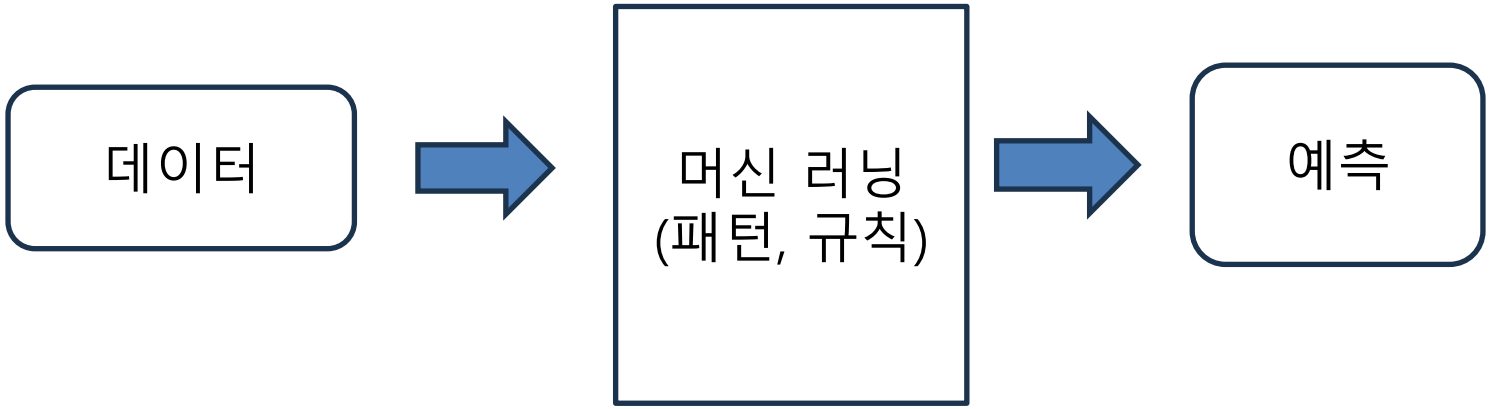
머신러닝(Machine Learning)

❖ **지도 학습**



머신러닝(Machine Learning)

❖ **비지도 학습**



머신러닝(Machine Learning)

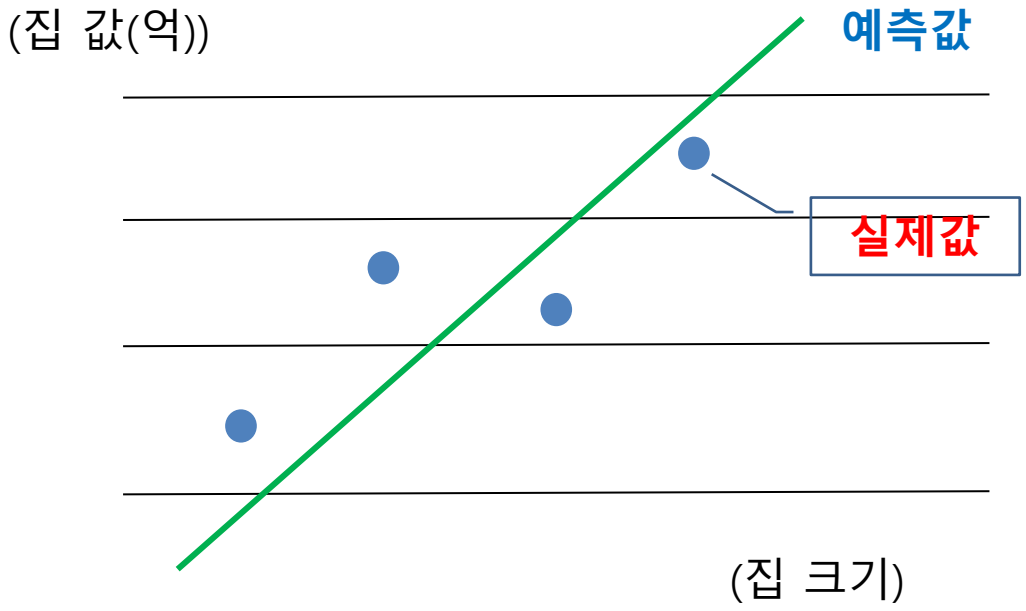
❖ 강화 학습

- 강화란 '어떤 것의 수준이나 정도를 높인다'는 의미로 인공지능의 수준을 높이는 것을 말한다.
- 에이전트가 환경과 상호작용하면서 보상 신호에 따라 최적의 정책을 찾아내는 방법

- 벽돌깨기 게임 – 딥마인드(구글) 회사
- 알파고(딥마인드) – 바둑을 학습한 인공지능
- 자율주행 자동차, 인공지능 로봇으로 확장되고 있음

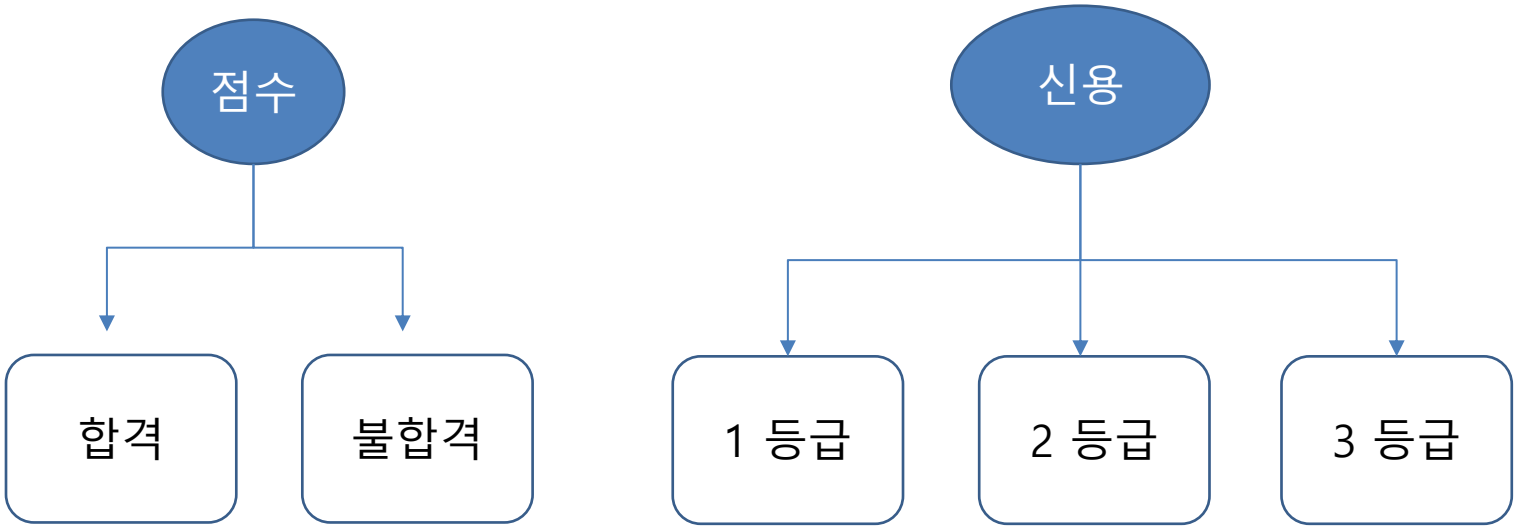
회귀(Regression)

- **부동산 예측 가격 선형 회귀**



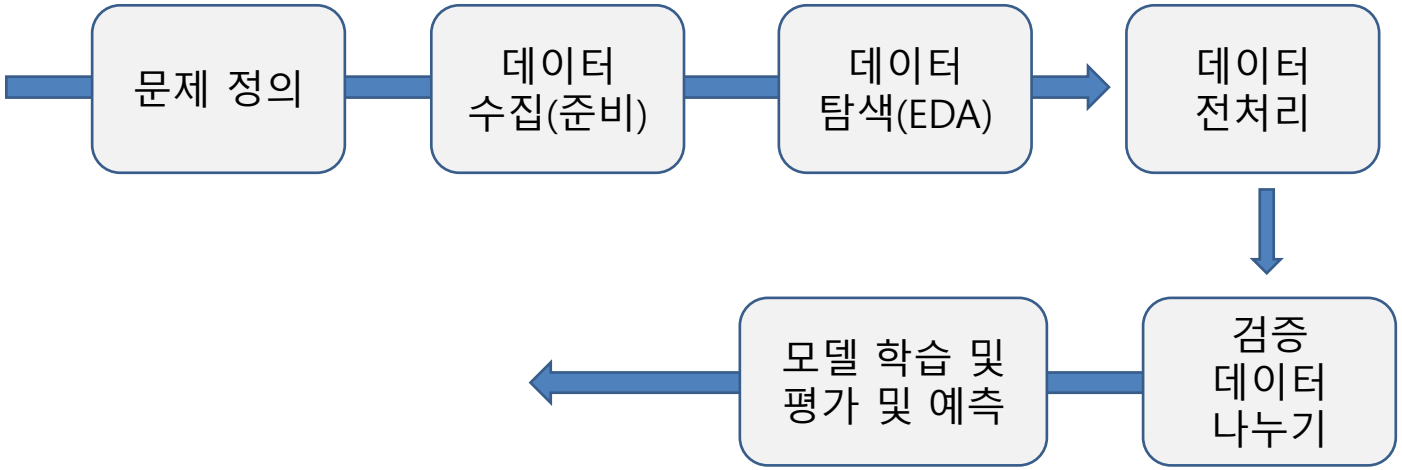
실제값과 예측값 사이의 오차를 최소화 하는 선을 찾는 것이 목적

분류(Classification)



머신러닝 프로세스

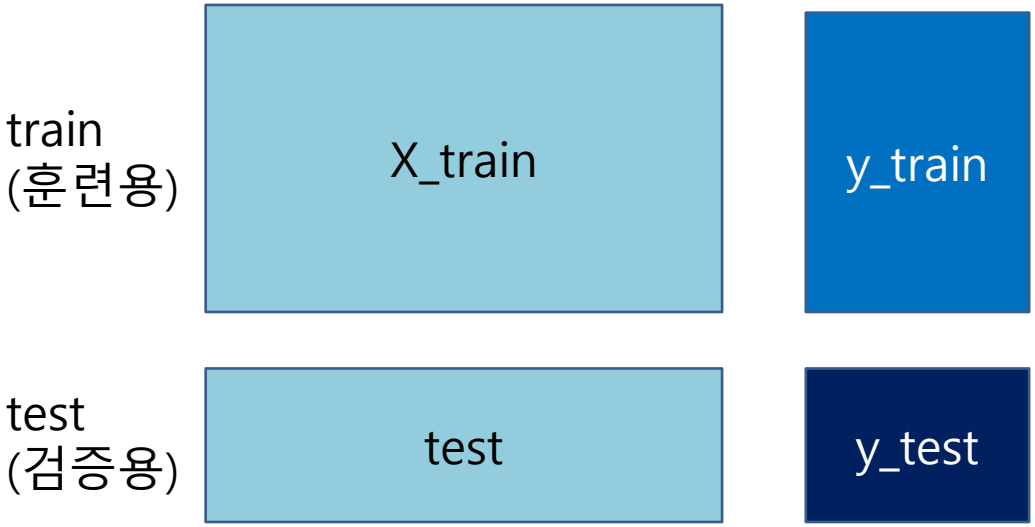
- 머신러닝 모델 예측 프로세싱



데이터 수집(준비)

- 데이터는 학습용과 검증으로 준비한다.
  - ✓ 훈련 : 검증 비율

    80 : 20 or 70 : 30



모델 학습 및 평가

- 모델 학습 및 평가

지도 학습	알고리즘	평가 지표
분류	sklearn.ensemble(모듈) RandomForestClassifier	sklearn.metrics(모듈) roc_auc_score, f1_score
회귀	sklearn.linear_model(모듈) LinearRegression sklearn.ensemble(모듈) RandomForestRegressor	sklearn.metrics(모듈) mean_squared_error mean_absolute_error

회귀 분석

- **회귀 분석**

  회귀 분석은 결과값(종속 변수)에 영향을 주는 요인(독립 변수)들 사이의 관계를 수학적으로 모델링하는 방법입니다.

종류	특징	예시
단순 선형 회귀	X가 1개, 관계를 직선으로 표현	공부 시간에 따른 점수 예측
다중 선형 회귀	X가 2개 이상, 복합적 영향 분석	면적, 위치, 연식에 따른 집값 예측
로지스틱 회귀	결과가 범주형(0 또는 1)인 경우	스팸 메일 여부(Yes/No) 판별

선형 회귀 분석

- **단순 선형 회귀 분석**

  독립 변수(X)가 하나일 때, 종속 변수(Y)와의 관계를 직선 형태로 분석하는 것입니다.

  - 수식: $y = \beta_0 + \beta_1 x + \epsilon$
  - 특징: 데이터의 분포를 가장 잘 나타내는 2차원 평면상의 직선을 찾습니다.
  - 예시: "공부 시간(X)"에 따른 "시험 점수(Y)" 예측. 변수가 시간 하나뿐이므로 관계가 직관적입니다.

선형 회귀 분석

- **다중 선형 회귀 분석**

  독립 변수(X)가 하나일 때, 종속 변수(Y)와의 관계를 직선 형태로 분석하는 것입니다.

  - 수식: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \epsilon$
  - 특징: 변수가 2개라면 3차원 공간에서 '평면(Plane)' 형태의 모델이 만들어지며, 그 이상일 경우 시각화는 어렵지만 수학적인 **'초평면(Hyperplane)'**을 형성합니다.
  - 예시: "집값(Y)"을 예측할 때 "면적(X1)", "역까지의 거리(X2)", "주변 학군(X3)" 등 여러 요인을 동시에 고려하는 경우입니다.

선형 회귀 분석

- **공부한 시간의 차이에 따른 시험 성적 예측하기**
  - 성적을 변하게 하는 요소를 x라 하고 이 x값에 따라 변하는 '성적'을 y라 하자.
  - 독립 변수(x) : 공부한 시간, 사교육비 지출, 학생의 지능지수 등
  - 종속 변수(y) : 성적
  - 어떤 변수가 다른 변수에 영향을 준다면 두 변수 사이에 선형관계가 있다.

선형 회귀 분석

- **공부한 시간의 차이에 따른 시험 성적 예측하기**

공부 시간	2시간	4시간	6시간	8시간
성적	81점	93점	91점	97점

x = {2, 4, 6, 8}

y = {81, 93, 91, 97}

직선의 방정식을 구한다.

$y = ax + b$(일차함수)

기울기 a와 절편 b를 알아야 함

최소 제곱법

- **최소 제곱법**
  - 기울기 a 구하기

$$a = \frac{(x - x\text{평균})(y - y\text{평균})\text{의 합}}{(x - x\text{평균})^2\text{의 합}}$$

공부 시간(x) 평균 : (2 + 4 + 6 + 8) ÷ 4 = 5

성적(y) 평균: (81 + 93 + 91 + 97) ÷ 4 = 90.5

기울기 a = 2.3

X의 편차(각 값과 평균과의 차이)를 제곱해서 합한 값을 분모로 놓고,

x와 y의 편차를 곱해서 합한 값을 분자로 놓으면 기울기가 나옴

최소 제곱법

- **통계 용어**

  - 편차(deviation)

    관측값에서 평균을 뺀 값

  - 분산 (variance)

    편차(관측값-평균)를 제곱하고 그것을 모두 더한 후
    전체 개수로 나눈것. 즉 편차의 제곱의 평균을 말한다.

  - 표준편차(standard deviation)

    분산의 제곱근, 즉 분산 값에 루트를 씌운것을 말하며
    stdev라고 함.

최소 제곱법

- **공부한 시간의 차이에 따른 시험 성적 예측하기**
  - y절편 b값 구하기

b = y의 평균 – (기울기 a x (x의 평균))

b = 90.5 – (2.3 x 5) = 79

최적의 직선(방정식)

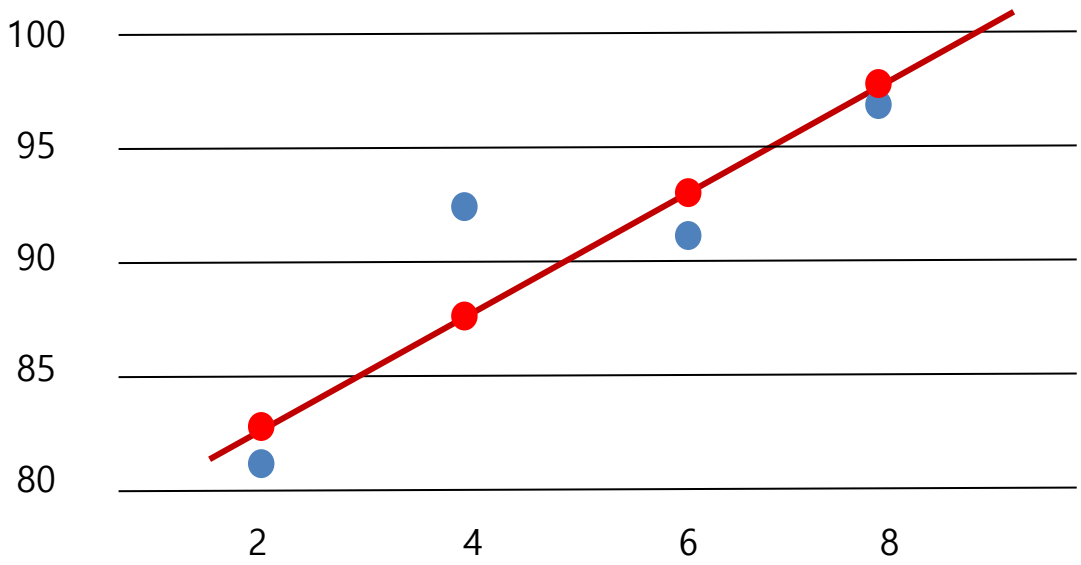
$y = 2.3x + 79$

공부 시간	2	4	6	8
성적	81	93	91	97
예측 값	83.6	88.2	92.8	97.4

최소 제곱법

- 공부한 시간의 차이에 따른 시험 성적 예측하기

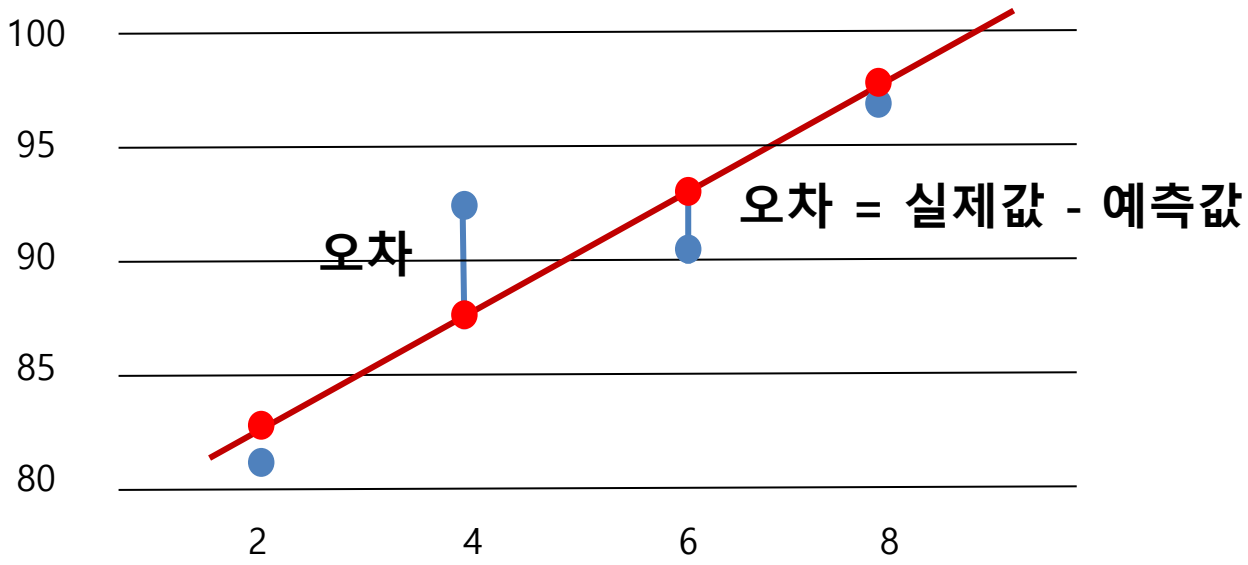
공부 시간	2	4	6	8
성적	81	93	91	97
예측 값	83.6	88.2	92.8	97.4



평균 제곱 오차

- **평균 제곱 오차**

  선형 회귀는 임의의 직선을 그어 이에 대한 평균 제곱 오차를 구하고, 이 값을 가장 작게 만들어주는 기울기(a)와 절편(b)을 찾아가는 작업이다.



평균 제곱 오차

- **성능 평가 - 평균 제곱 오차 / 평균 절대값 오차**

  - MSE(Mean Squared Error)
    - 평균 제곱오차
    - 오차의 제곱을 평균으로 나눈 것

$$MSE = \frac{\sum_{i=1}^{n}(y-\hat{y})^2}{n}$$

  - MAE(Mean Absolute Error)
    - 평균 절대값 오차
    - 오차의 차이를 절대값으로 변환한 뒤 합산

$$MAE = \frac{\sum |y-\hat{y}|}{n}$$

평균 제곱 오차

- **오차 = 실제값 – 예측값**

임의의 직선을 y = 3x + 76이라 하면

공부 시간	2	4	6	8
성적	81	93	91	97
예측 값	82	88	94	100
오차	**1**	**-5**	**3**	**3**

분모 : 1 + 25 + 9 + 9 = 44

MSE = 44 / 4 = 11

MAE = 1+5+3+3 = 12

구글 코렙

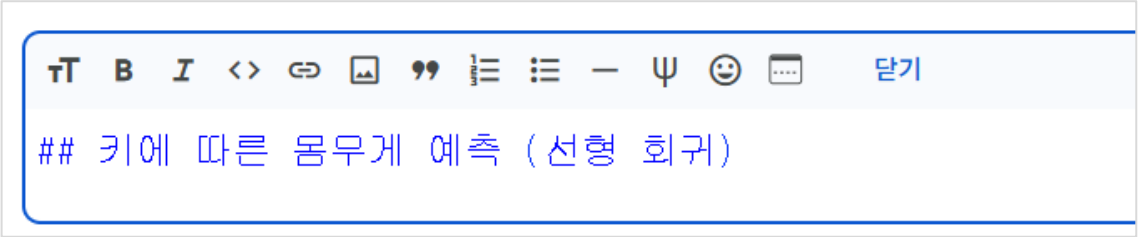
❖ 새 폴더 만들기

- 드라이브 > 내 드라이브
- + 신규 > 새폴더> 머신러닝

구글 코렙

❖ 텍스트 사용하기

- 파일 이름 : 선형회귀.ipynb
- 텍스트(버튼) > 문서의 제목을 만들때 사용



<편집화면>

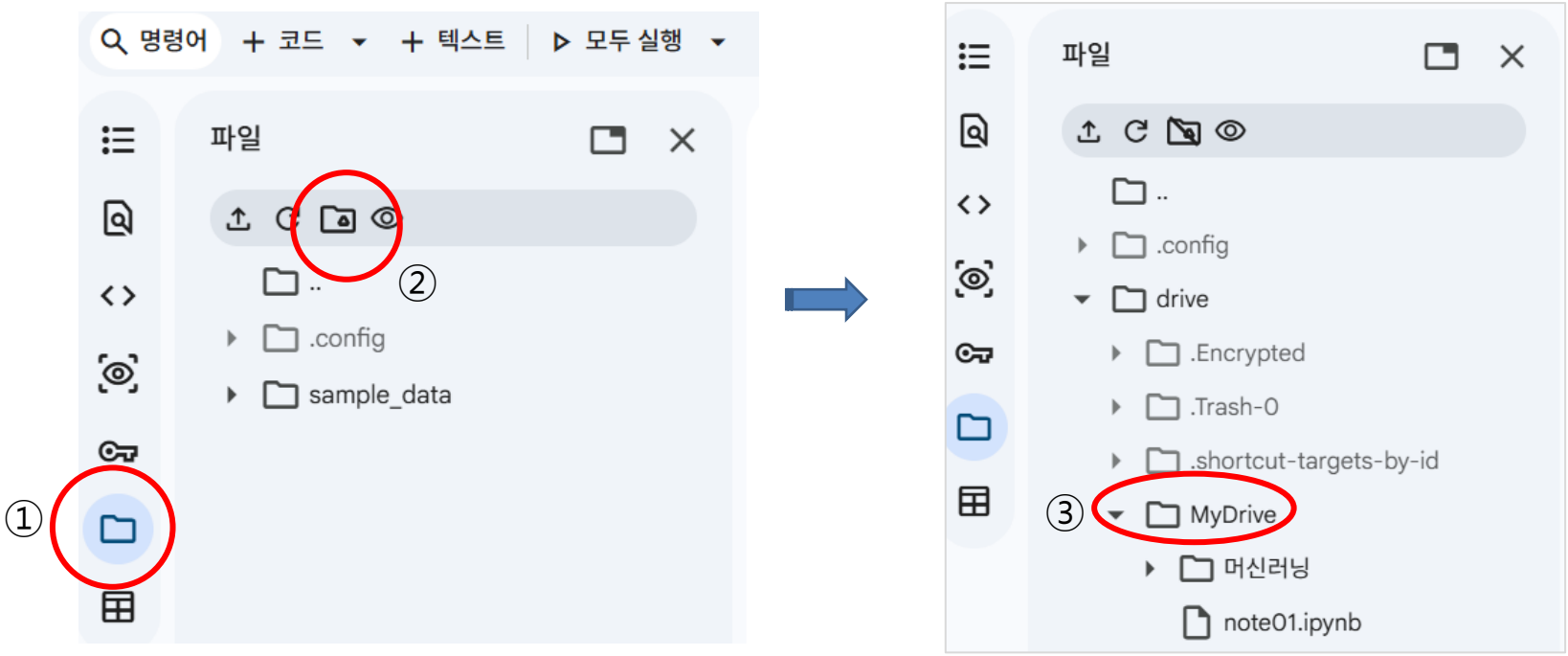
shift + enter

키에 따른 몸무게 예측 (선형 회귀)

<출력화면>

❖ 드라이브 마운트

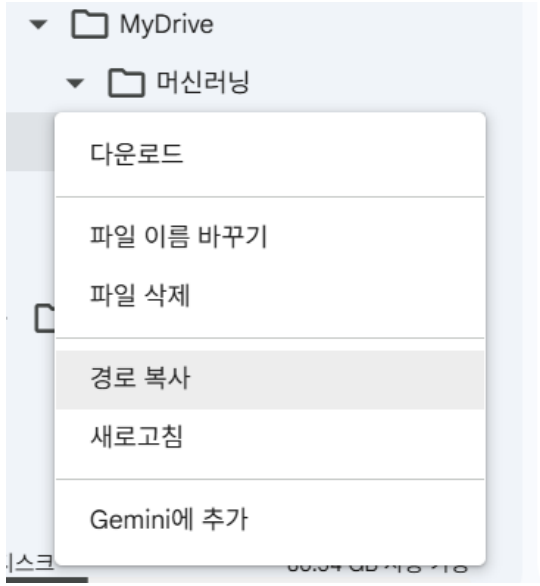
- 폴더 > 드라이브 마운트 > 드라이브 연결

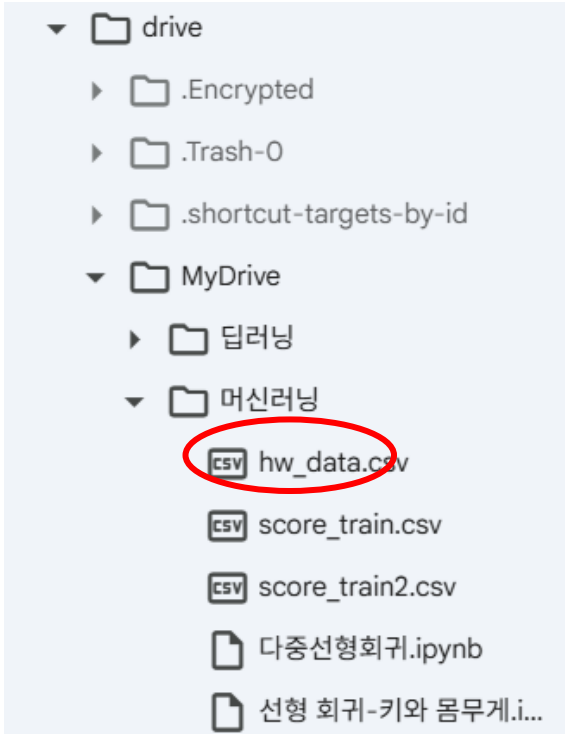


구글 코렙

❖ 훈련 데이터 업로드

- (...) 메뉴 > 업로드 > data_hw.csv





키에 따른 몸무게 예측

- **데이터 준비**

```
import pandas as pd

# 가상의 키와 몸무게 데이터 생성
data_hw = {
    "키": [150, 160, 170, 175, 165, 155, 172, 168, 174, 158,
           162, 173, 185, 159, 167, 163, 171, 169, 180, 161],
    "몸무게": [42, 50, 70, 65, 60, 48, 68, 60, 65, 40,
              54, 67, 89, 51, 65, 60, 69, 71, 85, 53]
}
# 데이터 프레임 생성
df_hw = pd.DataFrame(data_hw)
df_hw.head()

# csv 파일로 저장하기
df_hw.to_csv("/content/drive/MyDrive/Machine-Learning/hw_data.csv")
```

키에 따른 몸무게 예측

- **hw_data.csv**

hw_data.csv

1 to 10 of 20 entries Filter

키	몸무게
150	42
160	50
170	70
175	65
165	60
155	48
172	68
168	60
174	65
158	40

Show 10 per page 1 2

키에 따른 몸무게 예측

- **데이터 분할**

```
# csv 파일 읽기
df_hw = pd.read_csv("/content/drive/MyDrive/Machine-Learning/hw_data.csv")
df_hw.head()

# 변수 선택
X = df_hw[['키']]  #독립변수
y = df_hw['몸무게'] #종속변수

# 데이터 분할
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
            test_size=0.2, random_state=11)

print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(16, 1) (4, 1) (16,) (4,)
```

키에 따른 몸무게 예측

- **모델 학습 및 예측**

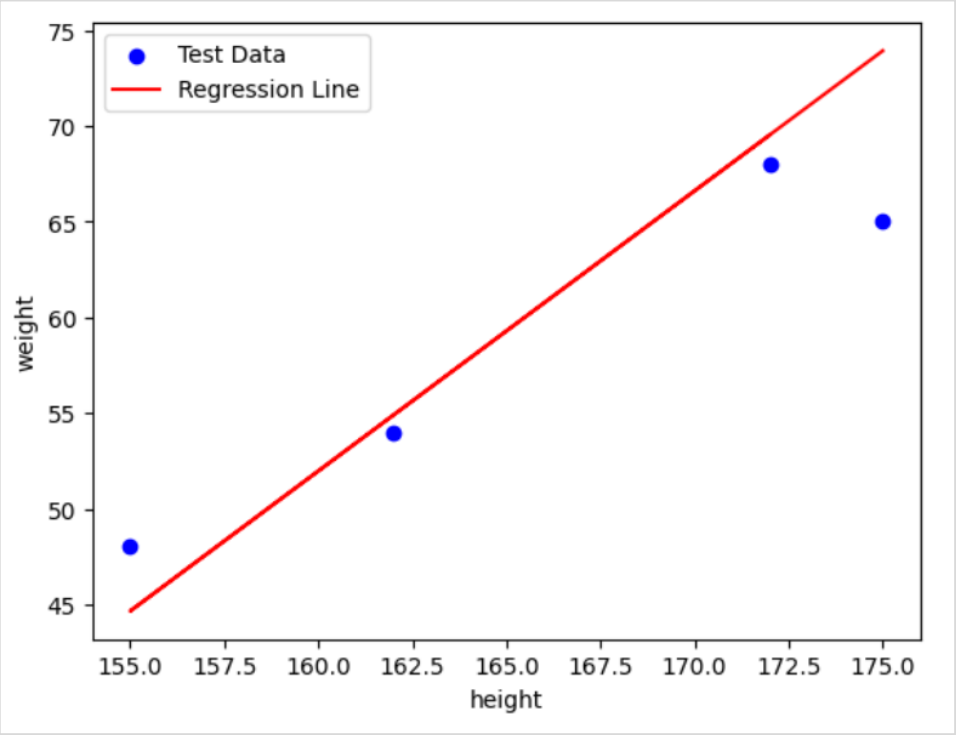
```python
# 선형 회귀 모델 학습 및 예측
from sklearn.linear_model import LinearRegression

# 모델 객체
model = LinearRegression()

# 모델 학습
model.fit(X_train, y_train)

# 모델 예측
y_pred = model.predict(X_test)
print("예측 결과:₩n", y_pred[:5])
```

```
예측 결과:
 [73.93695166 44.64699471 54.89847965 69.54345812]
```

키에 따른 몸무게 예측

- **테스트 데이터 모델 시각화**

```
import matplotlib.pyplot as plt

plt.scatter(X_test, y_test, color='blue', label='Test Data')
plt.plot(X_test, model.predict(X_test), color='red', label='Regression Line')
plt.xlabel('height')
plt.ylabel('weight')
plt.legend()
plt.show()
```



키에 따른 몸무게 예측

- **모델 성능 평가**

```python
from sklearn.metrics import mean_squared_error
import numpy as np

# 성능 평가
mse = mean_squared_error(y_test, y_pred)
print(f"평균제곱오차(MSE): {mse:.4f}")

rmse = np.sqrt(mse)
print(f"평균제곱근오차(RMSE): {rmse:.4f}")
```

```
평균제곱오차(MSE): 23.5753
평균제곱근오차(RMSE): 4.8554
```

키에 따른 몸무게 예측

- **모델 성능 평가 및 예측**

키에 따른 몸무게 예측에서 mse와 rmse 값의 성능을 평가해 줘

+ Gemini 2.5 Flash

모델의 성능은 평균제곱오차(MSE)와 평균제곱근오차(RMSE)로 평가됩니다.

- **평균제곱오차(MSE)**는 실제 값과 예측 값의 차이를 제곱하여 평균한 값입니다. MSE가 낮을수록 모델의 예측 정확도가 높다고 할 수 있습니다. 현재 모델의 MSE는 35.5926입니다.
- **평균제곱근오차(RMSE)**는 MSE에 제곱근을 취한 값입니다. 이는 오차의 크기를 원래 데이터의 단위와 동일하게 해석할 수 있도록 해주어 더 직관적입니다. RMSE가 낮을수록 모델의 예측 정확도가 높다고 할 수 있습니다. 현재 모델의 RMSE는 5.9660입니다.

이 값들은 모델의 성능을 나타내지만, 이 수치들이 좋고 나쁨을 판단하기 위해서는 예측하려는 '몸무게' 데이터의 범위나 다른 모델과의 비교가 필요합니다. 일반적으로 값이 작을수록 모델의 예측 오차가 작다고 해석할 수 있습니다.

키에 따른 몸무게 예측

- **몸무게 예측**

```
# 특정 데이터 예측
height = 165
weight = model.predict([[height]])
print(f"{height}cm 인 사람의 몸무게는 {weight[0]:.2f}kg 입니다.")
```

```
165cm 인 사람의 몸무게는 59.29kg 입니다.
```

키에 따른 몸무게 예측

- **몸무게 예측**

```
# 특정 데이터 예측
height = 165
weight = model.predict([[height]])
print(f"{height}cm 인 사람의 몸무게는 {weight[0]:.2f}kg 입니다.")
```

```
165cm 인 사람의 몸무게는 59.29kg 입니다.
```

Numpy(넘파이)

- **Numpy(Numeric Python)**

  - **대규모 수치 계산과 배열 연산을 빠르게 처리**하기 위한 파이썬 라이브러리입니다.
  - 과학 계산, 데이터 분석, 머신러닝(딥러닝)에 주로 활용됩니다.

수학 관련 함수	설명
np.sum([1, 2, 3])	합계(리스트, 튜플)
np.mean([1, 2, 3])	평균(리스트, 튜플)
np.square(n)	n의 제곱수
np.sqrt(n)	n의 제곱근

딥러닝(Deep Learning)

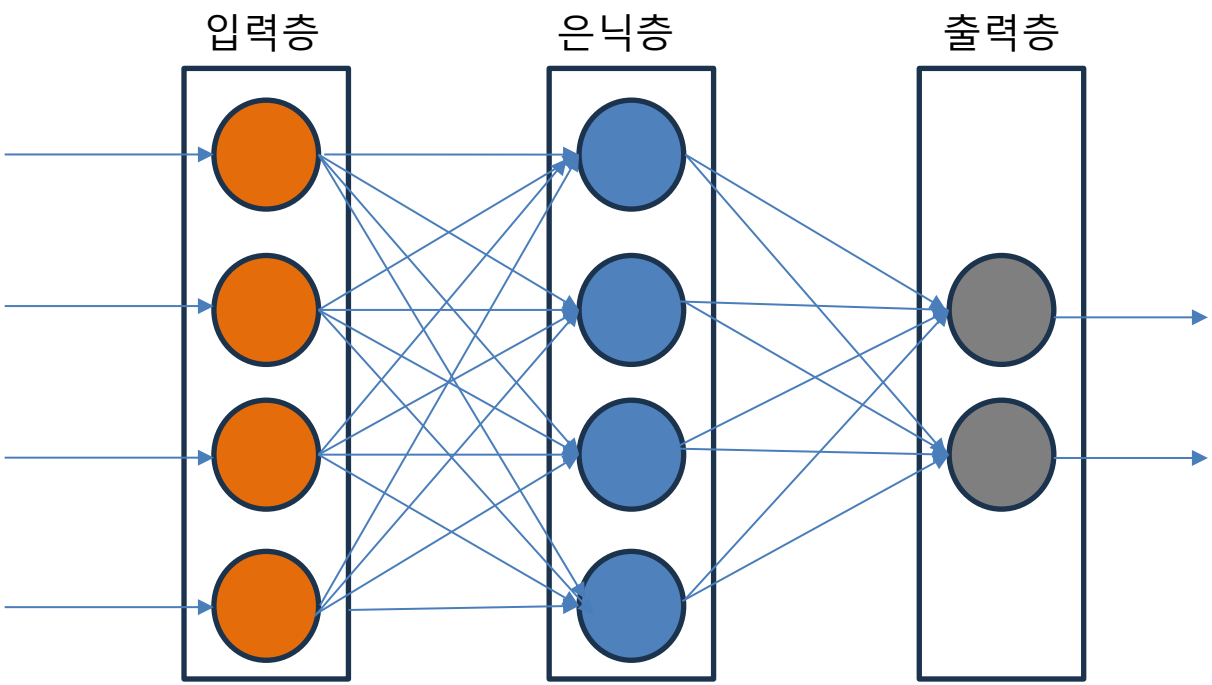
- **딥러닝이란?**

딥러닝(Deep Learning)은 인공 신경망을 기반으로 하는 머신러닝의 한 분야입니다.
사람의 뇌가 작동하는 방식과 유사하게, 여러 계층(layer)의 신경망을 통해 데이터를 학습하고 복잡한 패턴을 인식하여 예측이나 분류 등의 작업을 수행합니다.

입력층 –> 은닉층 –> 은닉층 –> 출력층

딥러닝(Deep Learning)

- **인공 신경망**
  - **입력층(input layer)** – 데이터를 입력받는 층
  - **은닉층(hidden layer)** – 입력층에서 들어온 데이터가 여러 신호로 바뀌어 출력층까지 전달됨
  - **출력층(output layer)** – 출력층에 어떤 값이 전달되었는냐에 따라서 예측 값이 결정됨



딥러닝(Deep Learning)

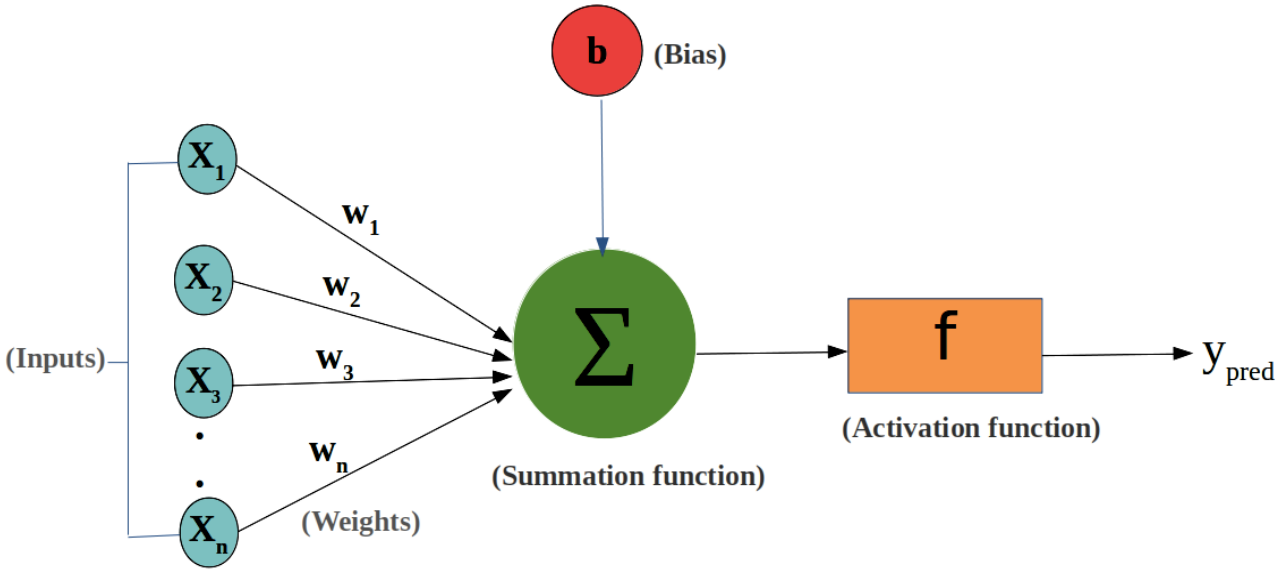
- **신호 전달 원리**

인공신경망에서는 단순하게 신호를 전달해 주는 것이 아니라 신호 세기를 변경해서 전달합니다.
신경망에서 중요한 용어 중 **가중치(weight)와 편향(bias)**이 있는데 이들이 바로 신호 세기를 변경하는 데 사용된다.
한 뉴런에서 다음 뉴런으로 전달되는 신호 세기는 가중치와 편향에 의해 결정됨

가중치란 그 값이 얼마나 중요한지 그렇지 않은지를 표현하기 위한 도구이고, **편향**은 한쪽으로 치우치는 값을 더할때 사용함

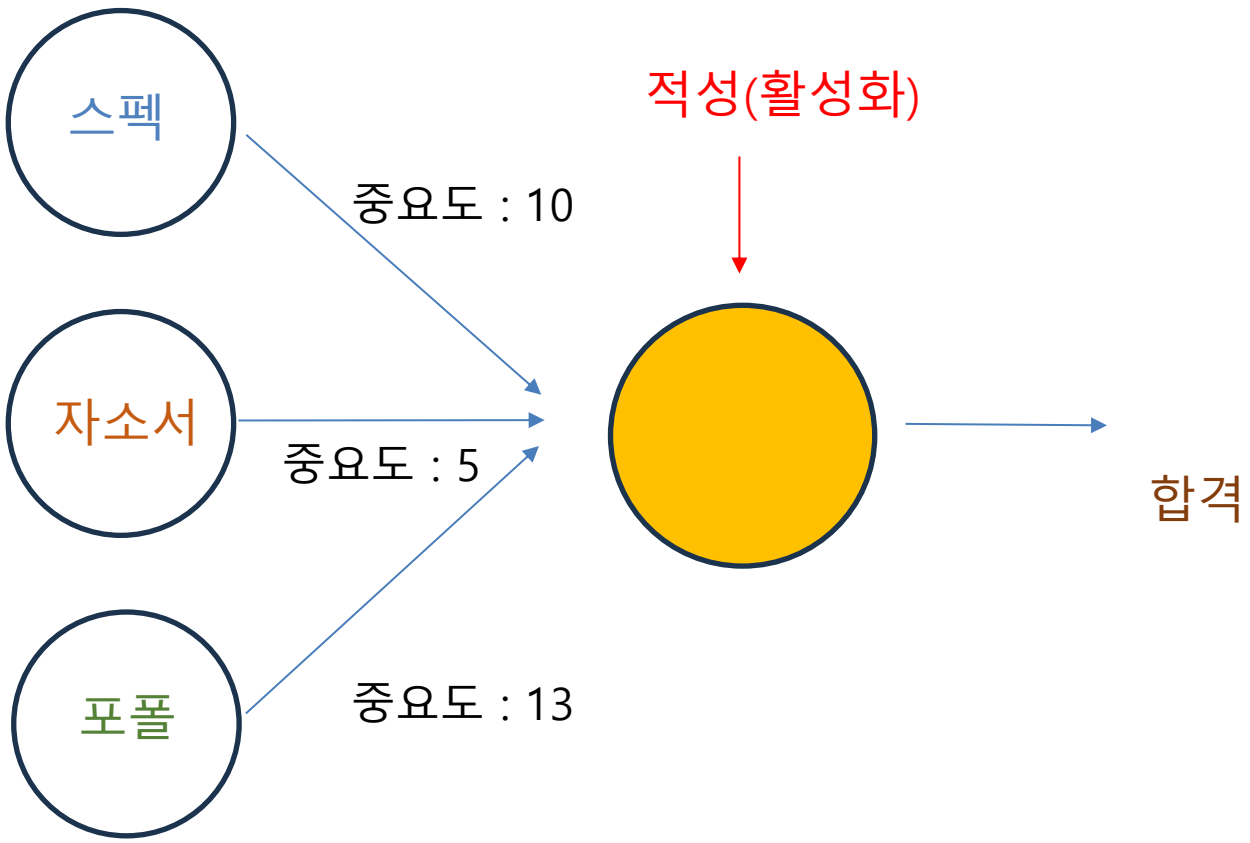
딥러닝(Deep Learning)

- 가중치와 편향



딥러닝(Deep Learning)

- **가중치와 편향**

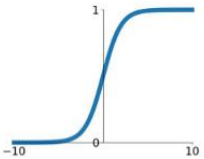


딥러닝(Deep Learning)

- **활성화 함수**
  - 뉴런을 활성화 할 필요가 있는지 없는 지 결정하는데 도움을 주는 함수
  - 비선형 문제를 해결하는데 중요한 역할을 한다

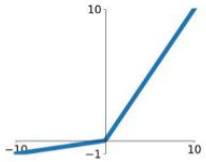
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



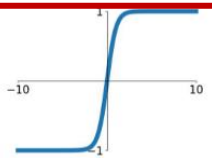
Leaky ReLU

$\max(0.1x, x)$



tanh

$\tanh(x)$

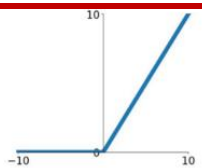


Maxout

$\max(w_1^T x + b_1, w_2^T x + b_2)$

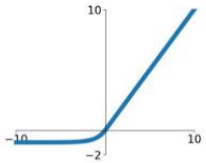
ReLU

$\max(0, x)$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



머신러닝과 딥러닝 비교

⚙️ 머신러닝 vs 딥러닝 한눈에 비교

구분	머신러닝	딥러닝
데이터	적어도 가능	많이 필요
특징 추출	사람이 설계	자동
계산량	적음	매우 큼
성능	보통	매우 높음

🎯 한 줄 요약

- **머신러닝**: 데이터로 규칙을 배우는 AI
- **딥러닝**: 신경망을 깊게 쌓아 더 똑똑하게 만든 머신러닝

딥러닝(Deep Learning)

- **손글씨 예측 문제**

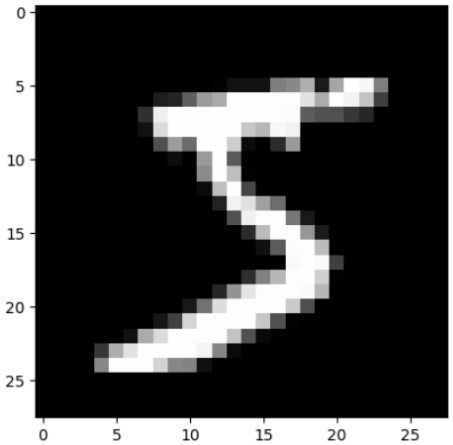
MNIST

미국 국립표준기술연구소에서 만든 손글씨 숫자 이미지를 혼합해서 만든 데이터 셋입니다.

머신러닝 및 딥러닝 분야에서 가장 널리 사용되는 데이터셋 중 하나입니다.

딥러닝(Deep Learning)

- **MNIST의 주요 특징**
  - **구성**: 0부터 9까지의 손글씨 숫자 이미지로 이루어져 있습니다.
  - **데이터 양**: 훈련 세트에 60,000개의 이미지, 테스트 세트에 10,000개의 이미 지가 포함되어 있습니다.
  - **이미지 크기**: 각 이미지는 28x28 픽셀의 흑백(회색조) 이미지입니다.



  - **용도**: 주로 **이미지 분류**, 특히 **숫자 인식 분야**에서 딥러닝 모델의 기본 성능 테스트 및 벤치마킹을 위해 활용됩니다. 간단하면서도 충분히 복잡하여 초보자들이 딥러닝을 학습하고 다양한 모델을 시도해보기 좋은 데이터셋입니다.

딥러닝(Deep Learning)

- **라이브러리 불러오기 및 데이터 준비**

```
from keras.datasets import mnist  #mnist
from keras.models import Sequential #모델 객체 생성
from keras.layers import Dense #은닉층
import matplotlib.pyplot as plt #시각화
import numpy as np  #수치 계산
```

```
# 데이터 준비
# y는 정답(미리 줘서 학습)
# 훈련용과 테스트 용으로 구분
(X_train, y_train), (X_test, y_test) = mnist.load_data()

X_train.shape, y_train.shape, X_test.shape, y_test.shape
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
((60000, 28, 28), (60000,), (10000, 28, 28), (10000,))
```

- **학습용 첫 데이터**

학습용 첫 데이터 보기

```
X_train[0]
```

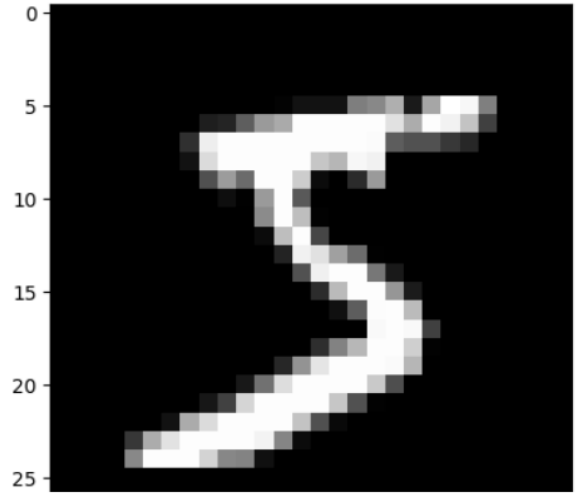
ndarray (28, 28) show data

```
print(y_train[0])
```

```
5
```

```
plt.imshow(X_train[0], cmap="gray") # 숫자를 이미지로 출력
```

```
<matplotlib.image.AxesImage at 0x7d006afc9160>
```



딥러닝(Deep Learning)

- **데이터 전처리**

```
# 인공신경망의 입력층에 들어가는 데이터는 한 줄로 만들어줌
X_train = X_train.reshape(60000, 784) #28*28=784
X_test = X_test.reshape(10000, 784)

# 0~255 ->데이터 정규화(0 ~ 1사이로 바꿈)
X_train = X_train / 255
X_test = X_test / 255
```

딥러닝(Deep Learning)

- **데이터 전처리**

```
X_train[0]
```

```
...
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.01176471, 0.07058824, 0.07058824,
       0.07058824, 0.49411765, 0.53333333, 0.68627451, 0.10196078,
       0.65098039, 1.        , 0.96862745, 0.49803922, 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.11764706, 0.14117647, 0.36862745, 0.60392157,
       0.66666667, 0.99215686, 0.99215686, 0.99215686, 0.99215686,
       0.99215686, 0.88235294, 0.6745098 , 0.99215686, 0.94901961,
       0.76470588, 0.25098039, 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.19215686, 0.93333333,
       0.99215686, 0.99215686, 0.99215686, 0.99215686, 0.99215686,
       0.99215686, 0.99215686, 0.99215686, 0.98431373, 0.36470588,
       0.32156863, 0.32156863, 0.21960784, 0.15294118, 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.07058824, 0.85882353, 0.99215686, 0.99215686,
```

- **모델 만들기**

```python
# 모델 객체 생성
model = Sequential()

# 입력층 - 은닉 2개층(128, 64) - 출력층
model.add(Dense(256, input_dim=784, activation='relu')) #입력층(활성화 함수)
model.add(Dense(128, activation='relu')) #은닉층
model.add(Dense(64, activation='relu')) #은닉층
model.add(Dense(10, activation='softmax')) #출력층(활성화 함수) - 숫자(0~9)

# 요약
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_s
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 256)	200,960
dense_1 (Dense)	(None, 128)	32,896
dense_2 (Dense)	(None, 64)	8,256
dense_3 (Dense)	(None, 10)	650

```
Total params: 242,762 (948.29 KB)
Trainable params: 242,762 (948.29 KB)
Non-trainable params: 0 (0.00 B)
```

- **모델 학습**

```
# 모델 컴파일(최적화 함수, 손실함수, 평가 지표)
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# 모델 학습(훈련데이터, 정답, 반복횟수, 배치)
# batch_size : 100개의 샘플을 하나의 배치로 묶어서 모델에 입력
model.fit(X_train, y_train, epochs=5, batch_size=100)
```

```
Epoch 1/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.9228 - loss: 0.2641
Epoch 2/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 5s 9ms/step - accuracy: 0.9702 - loss: 0.0992
Epoch 3/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 4s 7ms/step - accuracy: 0.9780 - loss: 0.0690
Epoch 4/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 4s 7ms/step - accuracy: 0.9842 - loss: 0.0495
Epoch 5/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - accuracy: 0.9877 - loss: 0.0378
<keras.src.callbacks.history.History at 0x782e021a6f60>
```

- **모델 성능 평가 및 예측**

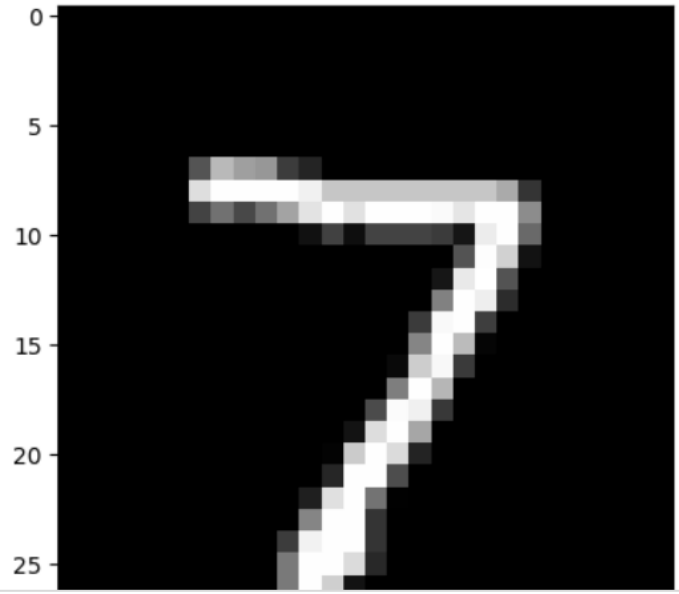
```python
# 모델 평가(테스트 데이터, 정답)
loss, acc = model.evaluate(X_test, y_test)
print('Test Loss:', loss)
print('Test Accuracy:', acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9734 - loss: 0.0950
Test Loss: 0.0950249433517456
Test Accuracy: 0.9733999967575073
```

```python
# 숫자 예측
model.predict(X_test[0].reshape(1, 784))
# 9.9990523e-01 : 확률이 가장 높으므로 예측한 숫자는 7임
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 120ms/step
array([[2.18188248e-07, 7.19453723e-08, 1.23337531e-05, 2.73820842e-05,
        2.19779409e-07, 6.47618563e-08, 1.11367825e-11, 9.99885798e-01,
        1.96096678e-07, 7.37322262e-05]], dtype=float32)
```

- **모델 성능 평가 및 예측**

```
# 예측한 숫자
pred_number = np.argmax(model.predict(X_test[0].reshape(1, 784)))
print(pred_number) #7

# 숫자 이미지 출력 - 출력시엔 이차원 형태로 변환
plt.imshow(X_test[0].reshape(28, 28), cmap='gray')
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step
7
<matplotlib.image.AxesImage at 0x782dfed748f0>
```